



SISTEMAS BIOINSPIRADOS

*MAGÍSTER EN INTELIGENCIA
ARTIFICIAL
AGOSTO 2026*

PROFESOR: RICARDO CONTRERAS A.

MODELOS PARA SISTEMAS DISCRETOS

- Redes de Petri.
- Autómatas celulares.

CARACTERÍSTICAS

- Redes de Petri
 - Modela eventos, recursos y concurrencia
 - A través de un flujo de *tokens*
 - *Su estructura es un grafo bipartito*
- Autómata Celular
 - Modela espacio y evolución local
 - A través de una grilla de celdas que cambian su estado
 - Su estructura es una grilla regular 1D, 2D o 3D (cada celda es una máquina de estados)

○ S E A

- Petri: lógica de procesos y recursos.
- Autómata celular: lógica de espacio y propagación.

REDES DE PETRI

- Son una herramienta gráfica y matemática para la descripción y el estudio de sistemas de procesamiento de información, que se caracterizan por ser concurrentes, asíncronas, distribuidas, paralelas, no deterministas y/o estocásticas.
- Origen: 1962
 - Facultad de Matemáticas y Física, Universidad Técnica de Darmstadt
 - “Comunicación con Autómatas”, Carl Adam Petri

REDES DE PETRI

- Es el modelo más antiguo y conocido para modelar procesos.
- Una red de Petri es un grafo dirigido $RP=(N,A)$, donde el conjunto de nodos está dividido en lugares y transiciones. $N= L \cup T$, y los arcos conectan solamente lugares a transiciones y transiciones a lugares.
- Dada una transición t , la premisa y la consecuencia de t son los siguientes conjuntos de lugares:
 - Premisa: $\{l/ \text{ existe un arco de } l \text{ a } t\}$
 - Consecuencia: $\{l/ \text{ existe un arco de } t \text{ a } l\}$

- Para describir el comportamiento dinámico de una Red de Petri, se consideran **redes marcadas**, es decir, redes en donde cada lugar está (o puede estar) marcado por un número de tokens.
- En una red marcada, una transición está activada cuando todos los lugares en su premisa tienen al menos un **token**.
- Una transición activada puede dispararse cambiando el marcado de la red.
- La actividad dinámica de una red marcada viene dada por los disparos de sus transiciones activadas. En general habrá más de una transición activada, lo que permite usar las Redes de Petri para modelar también procesos no deterministas.

- Debido a su generalidad, los modelos basados en Redes de Petri se vuelven muy grandes. Por ello, normalmente es necesario añadir modificaciones o restricciones cuando se usan en una aplicación particular.

AREAS DE APLICACIÓN

- Evaluación de rendimiento
- Programación paralela y concurrente
- Sistemas tolerantes a fallas
- Redes de área local
- Protocolos de comunicación
- Sistemas de control industrial
- Lenguajes formales
- ...

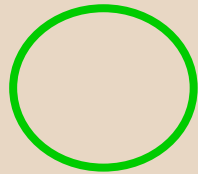
DEFINICIÓN FORMAL

- Una Red de Petri es una 5-tupla $RP=(L, T, I, P, M_0)$, donde:
 - $L = \{l_1, l_2, \dots, l_m\}$, conjunto finito, no vacío, de lugares
 - $T = \{t_1, t_2, \dots, t_n\}$, conjunto finito, no vacío, de transiciones
 - $I = L \times T \rightarrow \mathbb{N}$, función de incidencia previa
 - $P = T \times L \rightarrow \mathbb{N}$, función de incidencia posterior
 - $M_0: L \rightarrow \{1, 2, \dots\}$, es la marca inicial
 - $L \cap T = \emptyset$ y $L \cup T \neq \emptyset$

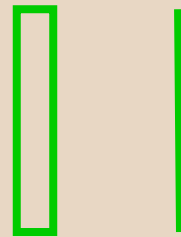
REPRESENTACIÓN GRÁFICA

- Una red de Petri es una clase particular de grafo dirigido junto con un estado inicial llamado marca inicial, M_0 .
- El grafo N de una red de Petri es bipartito, ponderado y dirigido, formado por dos clases de nodos: los lugares y las transiciones.
- En una representación gráfica, los lugares se dibujan como círculos y las transiciones como barras o cajas.
- Los arcos van de un lugar a una transición o desde una transición a un lugar y se rotulan con sus pesos. Un arco etiquetado con un peso k puede interpretarse como el conjunto de k arcos paralelos. El peso unitario suele omitirse.

- Una marca (estado) asigna a cada lugar un entero no negativo. Si un marcado asigna al lugar l el entero k , decimos que l está marcado con k tokens. Gráficamente, se colocarán k marcas en el lugar l .



Lugar



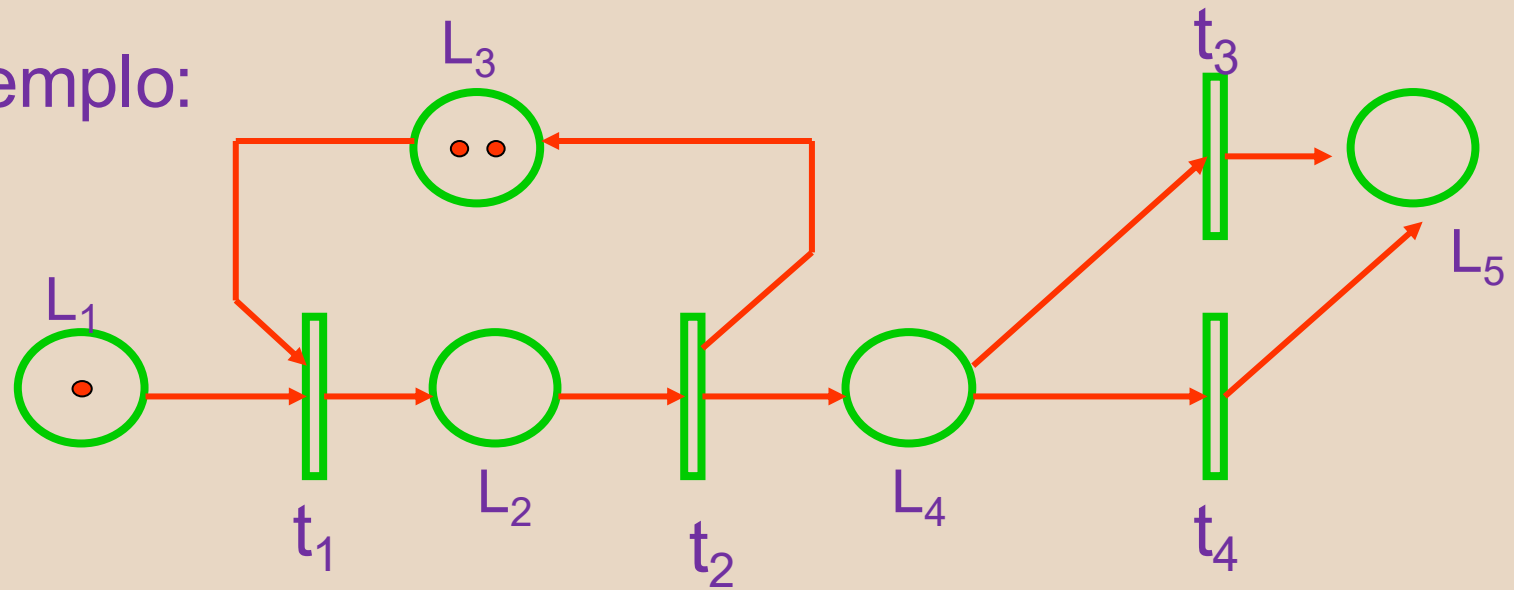
Transición

REPRESENTACIÓN MATRICIAL

- Una Red de Petri junto con su marcado se representa matricialmente por medio de dos matrices.
- Sea $|L| = n$, número de lugares de la red, y
- Sea $|T| = m$, número de transiciones de la red

- Se denomina matriz de incidencia previa a la matriz $C^- = [c_{ij}^-]_{n \times m}$, donde c_{ij}^- es el peso del arco del lugar i a la transición j .
- Se denomina matriz de incidencia posterior a la matriz $C^+ = [c_{ij}^+]_{n \times m}$ donde c_{ij}^+ es el peso del arco la transición j al lugar i .
- Los lugares numeran las filas (i) y las transiciones las columnas (j), y cada elemento (i,j) expresa la incidencia que tiene el lugar i sobre la transición j .
- La representación matricial se puede simplificar a través de una única matriz C , denominada matriz de incidencia $C = C^+ - C^-$

Ejemplo:



$$C^- = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 1 & 0 & 0 & 0 \\ 0 & 0 & 1 & 1 \\ 0 & 0 & 0 & 0 \end{bmatrix} \quad C^+ = \begin{bmatrix} 0 & 0 & 0 & 0 \\ 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 1 \end{bmatrix}$$

$\xrightarrow{t_i}$
 $\downarrow L_i$

Y LA MATRIZ DE INCIDENCIA ES:

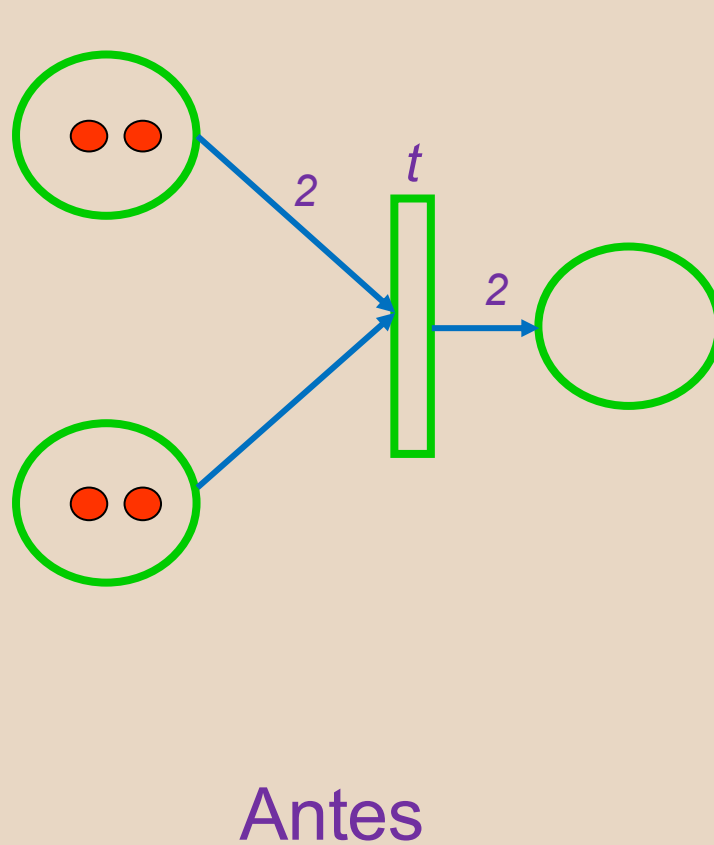
$$C = \begin{bmatrix} -1 & 0 & 0 & 0 \\ 1 & -1 & 0 & 0 \\ -1 & 1 & 0 & 0 \\ 0 & 1 & -1 & -1 \\ 0 & 0 & 0 & 0 \end{bmatrix}$$

CONCEPTOS DINÁMICOS

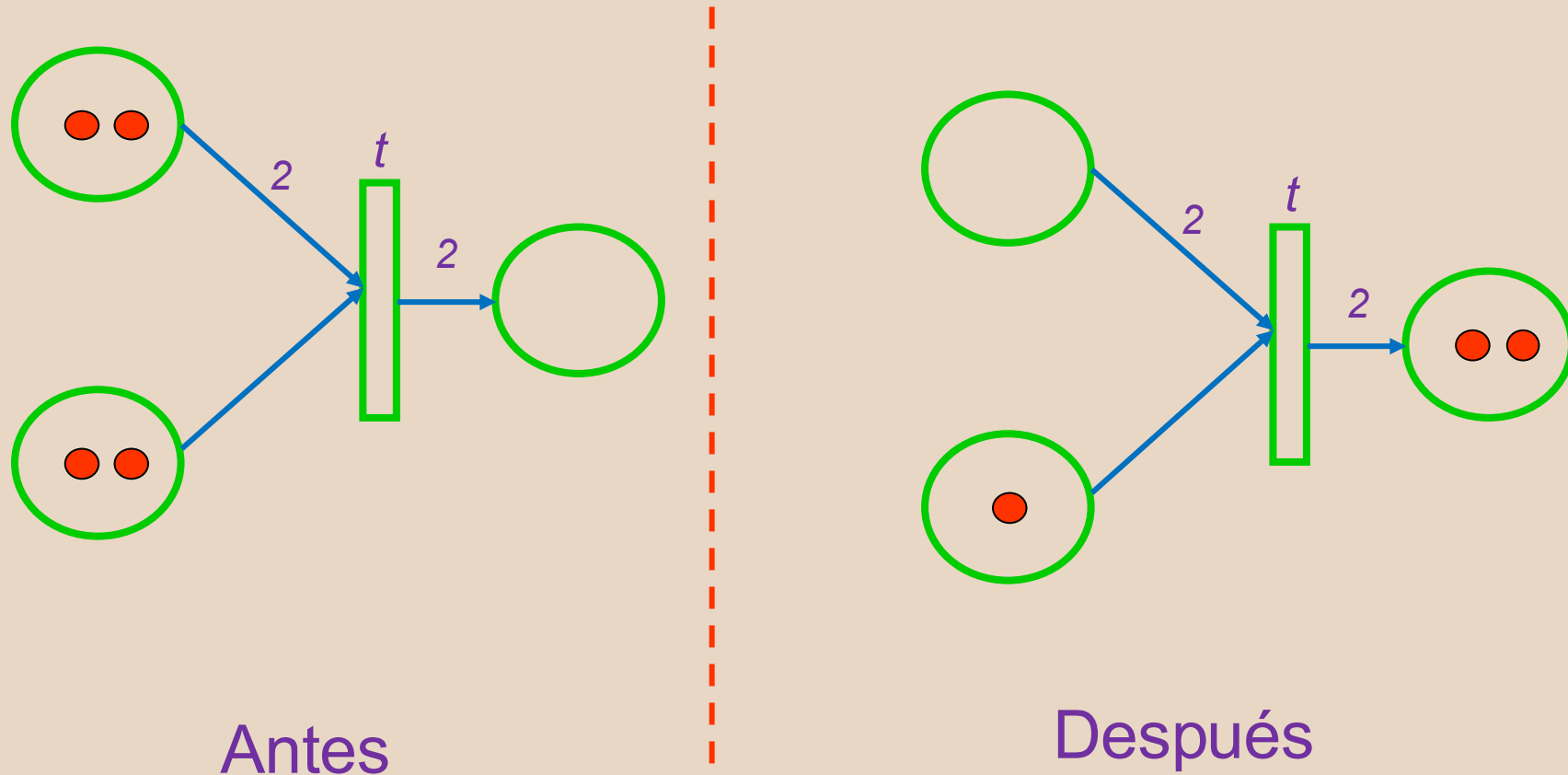
- El comportamiento de muchos sistemas puede describirse en términos de estados y sus cambios. Para simular el comportamiento dinámico de un sistema, un estado o marca en la Red de Petri se cambia de acuerdo a las siguientes reglas de transición:

- Una transición t se dice que está **habilitada** si cada lugar de entrada l de t es marcado con, al menos, $w(l,t)$ tokens, donde $w(l,t)$ es el peso del arco de l a t .
- Una transición habilitada puede o no puede dispararse (dependiendo de si el evento realmente tiene lugar o no).
- El disparo de una transición habilitada t elimina $w(l,t)$ tokens de cada lugar de entrada l de t y añade $w(l,t)$ tokens a cada lugar de salida l de t , donde $w(l,t)$ es el peso del arco de l a t .

ENTONCES, UNA RED DE PETRI ES UNA ESTRUCTURA
MATEMÁTICA DOTADA DE UNA PROPIEDAD DINÁMICA
(EL MARCADO)



ENTONCES, UNA RED DE PETRI ES UNA ESTRUCTURA MATEMÁTICA DOTADA DE UNA PROPIEDAD DINÁMICA (EL MARCADO)



INTERPRETACIÓN DE UNA RED DE PETRI

- Para que una Red de Petri pueda representar un sistema hace falta asociarle una interpretación a cada uno de sus elementos.
- Una interpretación consiste en darle un significado (físico) a:
 - Las condiciones necesarias para el disparo de una transición
 - Las acciones generadas por la evolución del marcado

EJEMPLO. AREA DE MODELACIÓN

- Los lugares representan condiciones y las transiciones eventos.
- Una transición tiene un cierto número de lugares de entrada y salida que representan las pre-condiciones y post-condiciones del evento.
- La presencia de un **token** en un lugar se interpreta como que la condición asociada al lugar es cierta.

INTERPRETACIONES USUALES:

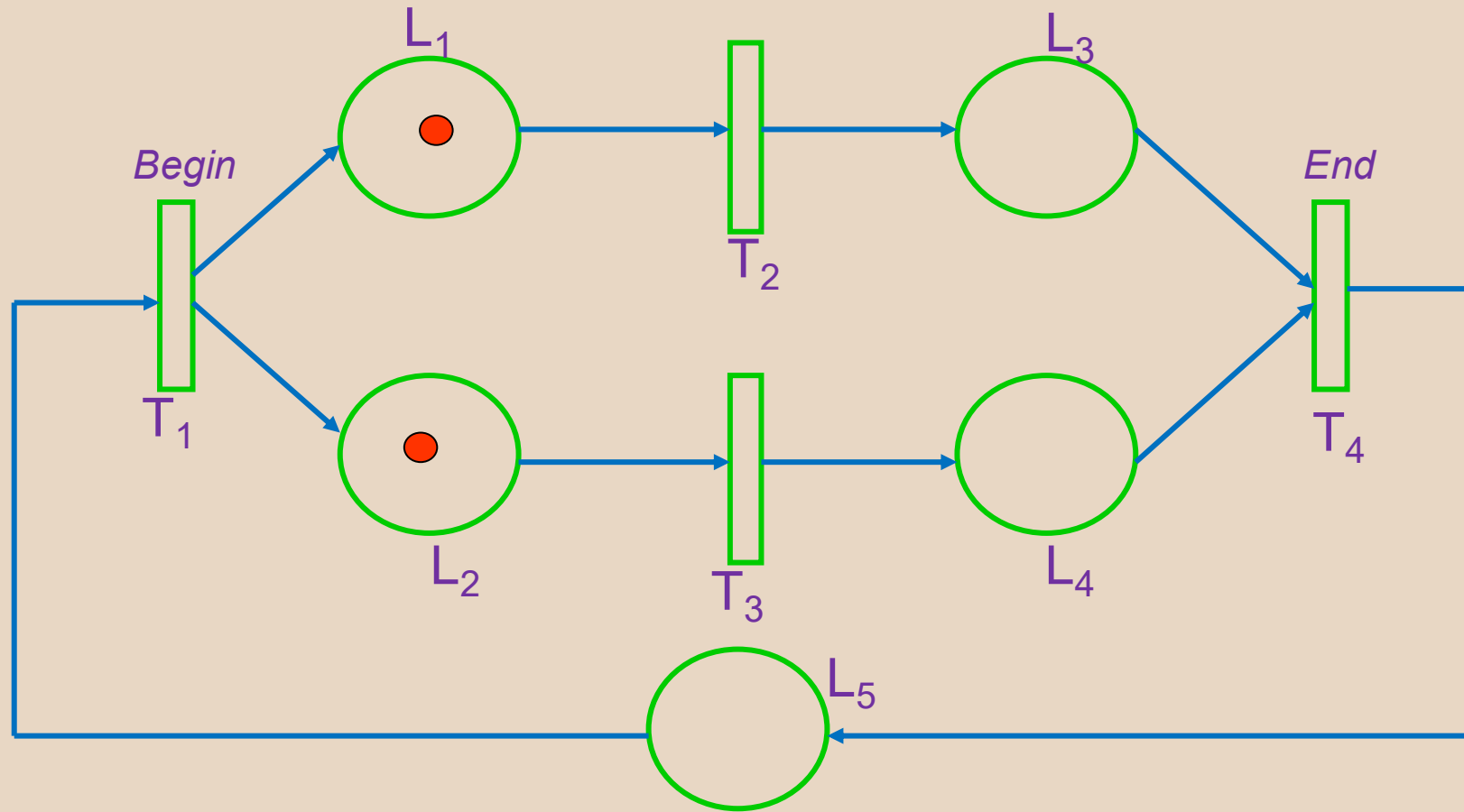
*Lugares de
entrada*

Transición

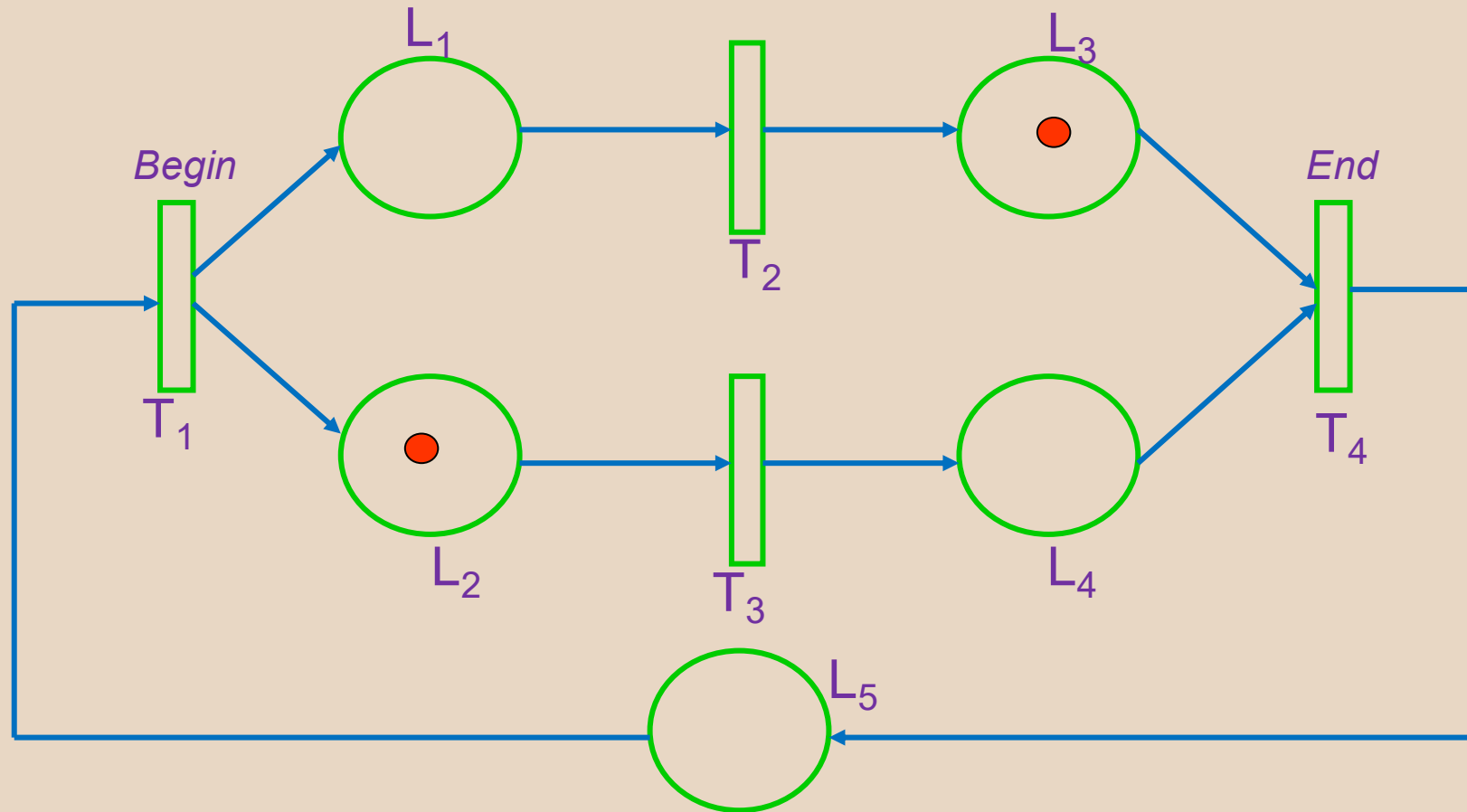
Lugares de salida

<i>Pre-condiciones</i>	<i>Evento</i>	<i>Post-condiciones</i>
<i>Datos de entrada</i>	<i>Paso de computación</i>	<i>Datos de salida</i>
<i>Señales de entrada</i>	<i>Procesador de señales</i>	<i>Señales de salida</i>
<i>Recursos necesarios</i>	<i>Tarea</i>	<i>Recursos liberados</i>
<i>Condiciones</i>	<i>Cláusulas lógicas</i>	<i>Conclusiones</i>

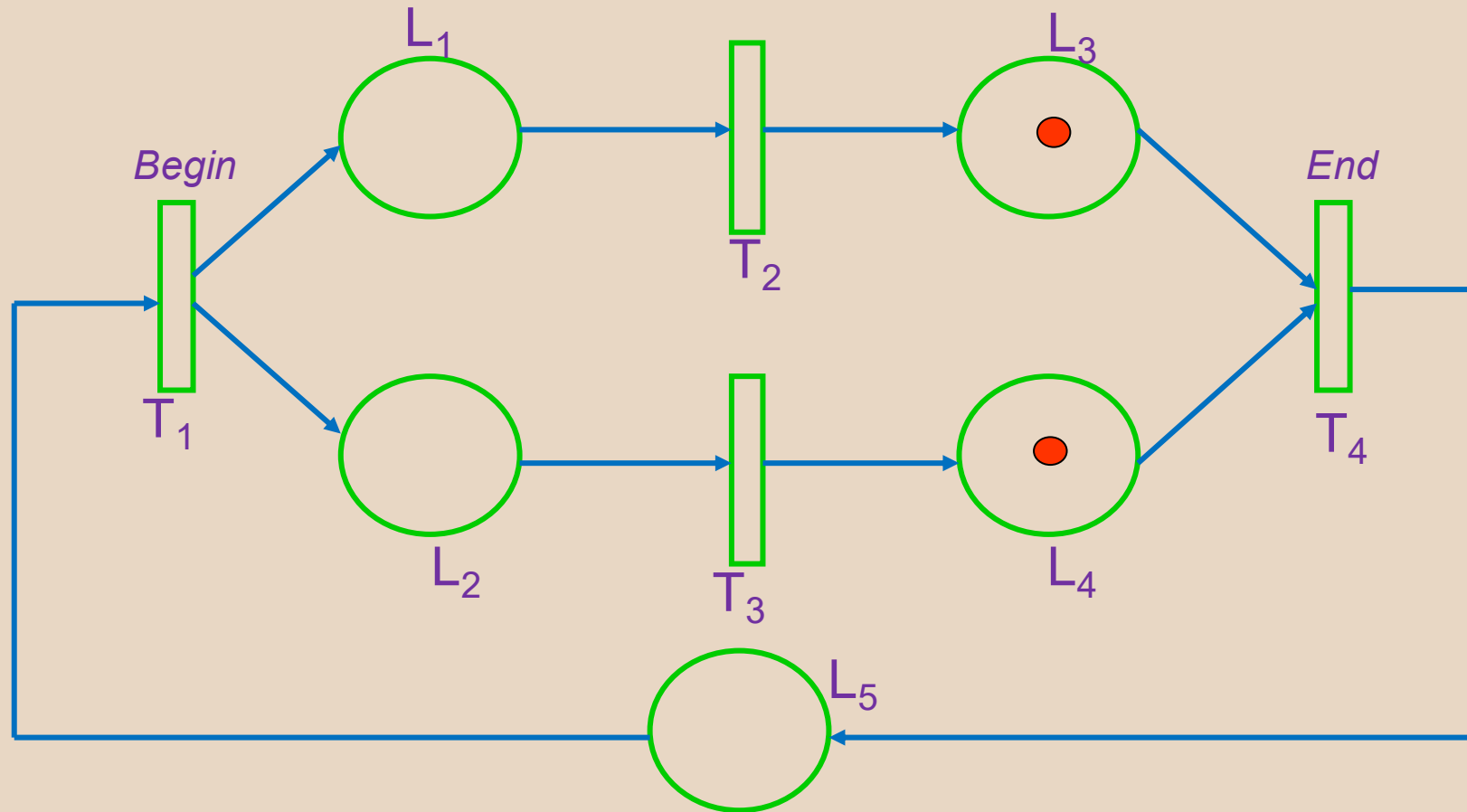
EJEMPLO. ACTIVIDADES PARALELAS



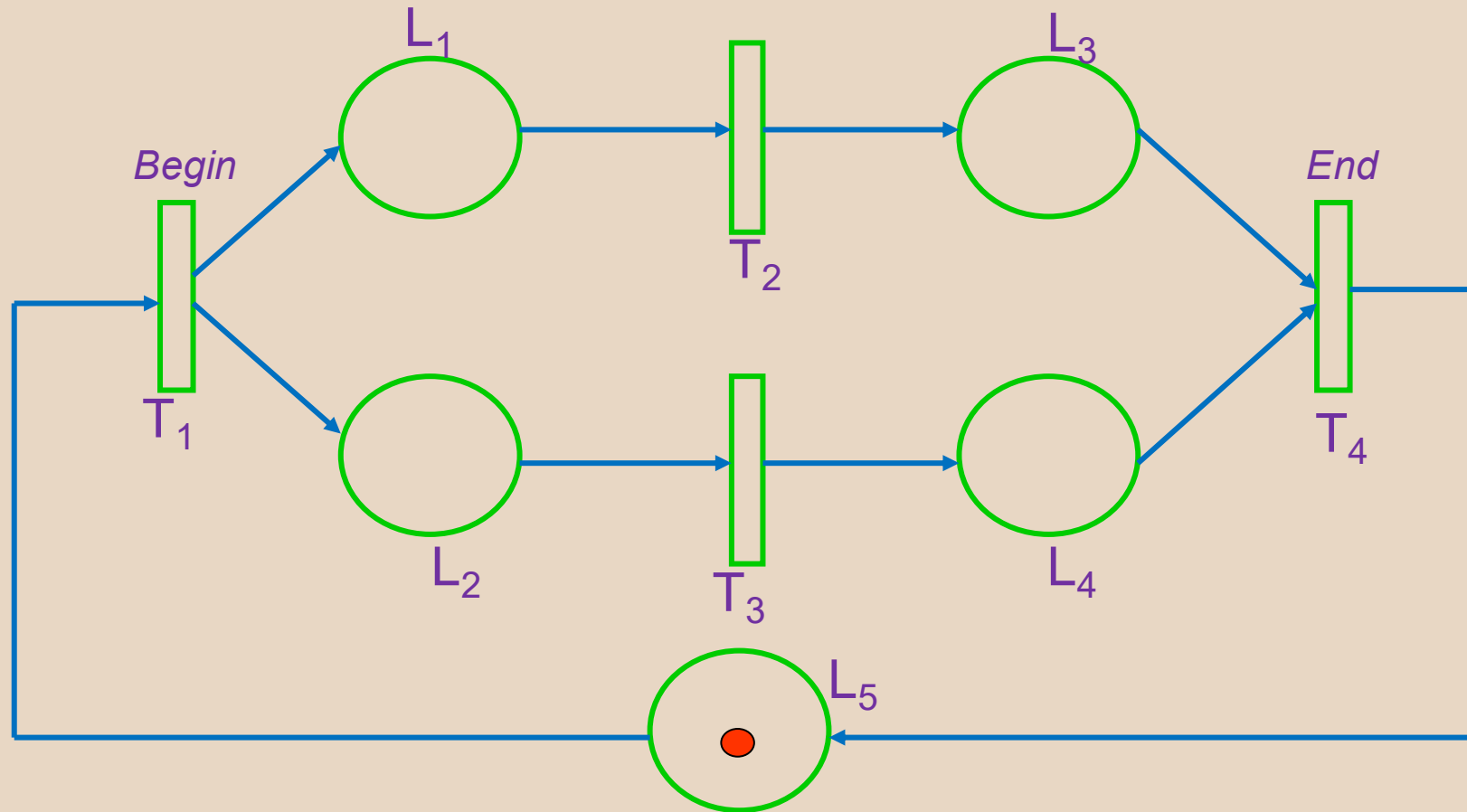
EJEMPLO. ACTIVIDADES PARALELAS



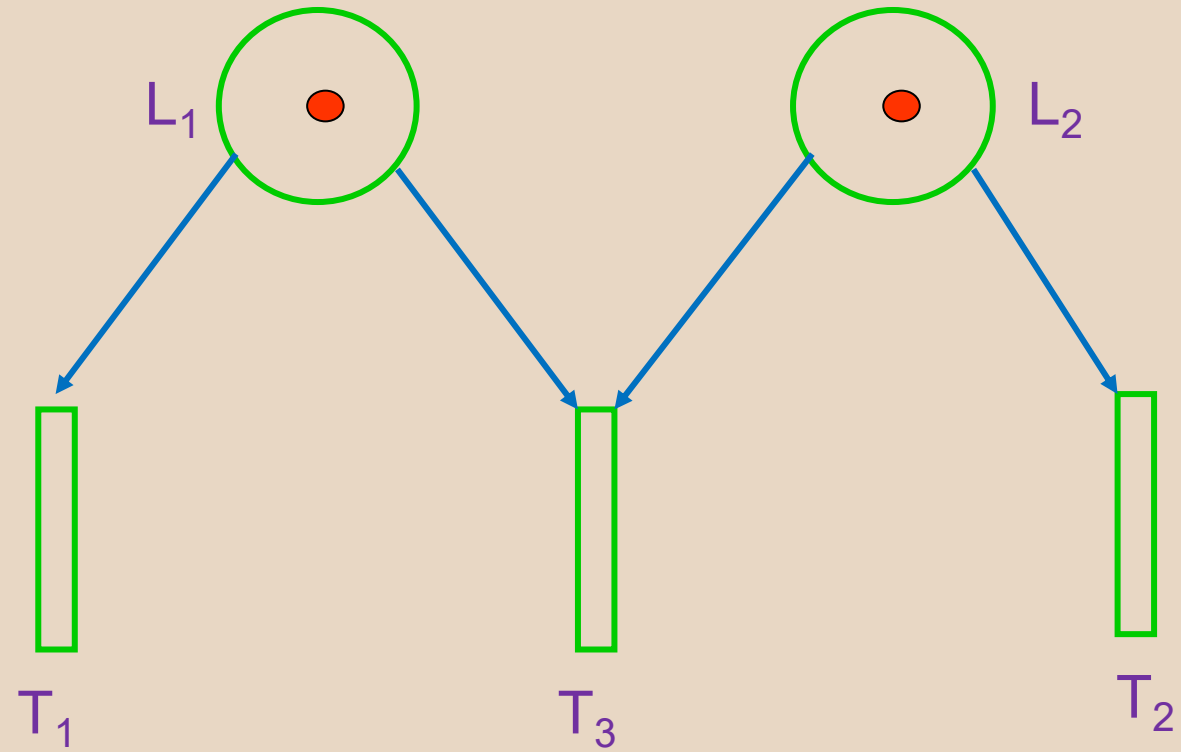
EJEMPLO. ACTIVIDADES PARALELAS



EJEMPLO. ACTIVIDADES PARALELAS



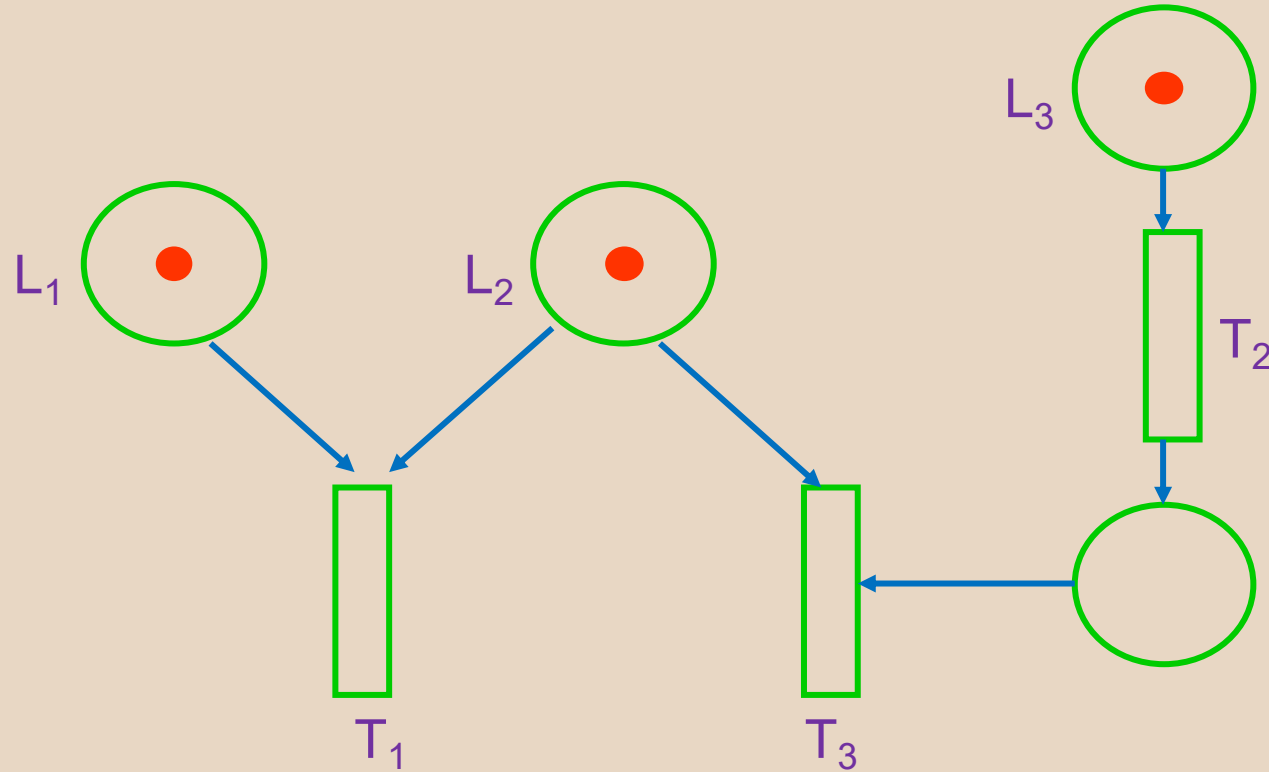
- Las Redes de Petri marcadas permiten representar concurrencia, pero no decisiones (conflictos).
- Dos eventos, e_1 y e_2 están en conflicto si e_1 o e_2 pueden ocurrir, pero no ambos; y son concurrentes si pueden ocurrir en cualquier orden sin conflictos.
- En una situación donde el conflicto y la concurrencia se pueden dar, se dice que existe una confusión.



T_1 y T_2 son concurrentes

T_1 y T_2 están en conflicto con T_3

CONFUSIÓN ASIMÉTRICA



T_1 es concurrente con T_2 , pero estará en conflicto con T_3 si T_2 dispara primero

AUTÓMATAS CELULARES (AC)

- Son sistemas dinámicos, discretos, que involucran reglas simples, determinísticas.
- Consisten de estructuras en general extensas, con variables indicando lugares denominados células.

Máquinas universales

- John von Neumann se interesó en la teoría de los autómatas autoreproductivos
- Ulam sugirió a von Neumann el uso de “espacios celulares”



Autómata celular – Máquinas universales

Von Neumann se dio cuenta de que la biología ofrecía el sistema de procesamiento de información más poderoso disponible y se preguntaba si era posible construir una máquina que produjese máquinas más complejas que ellas mismas. Máquinas que crearan otras máquinas, que a su vez se reproducirían en un bucle infinito.

Máquinas replicables, algo que él llamó Kinematon.

Inteligencias artificiales capaces de aprender por sí mismas, evolucionar, ¿lo están haciendo?.

Para von Neumann, el cerebro humano funcionaba igual que MANIAC, igual que un supercomputador, que podría descifrarse. En un libro no terminado, afirmaba que los computadores y los seres humanos son, sencillamente, “diferentes clases de autómatas”.

MANIAC: Mathematical Analyzer, Numerical Integrator and Computer

QUÉ ES UN AUTÓMATA CELULAR

- Un concepto desarrollado por Von Neumann, Ulam and Burk a fines de los 40's y 50's; (Autómata mecánico auto-reproducible)
- Juego de la Vida de Conway

Es un objeto matemático definido como:

- Un espacio celular n-dimensional e infinito, que consiste de celdas de igual tamaño.
- Las celdas están en uno, de un número discreto de estados.
- Las celdas cambian de estado como resultado de aplicar una regla de transición.
- Una regla de transición se define en función de los estados de las celdas que son parte de una vecindad.
- El tiempo avanza en pasos discretos. Todas las celdas cambian de estado simultáneamente.

COMO OPERA UN AUTOMATA CELULAR

Un automata celular monodimensional (AC) consiste de dos conjuntos de elementos: una fila de *celdas* y un conjunto de *reglas*.

Cada una de las celdas puede estar en uno de varios estados. El número de estados posibles depende del automata. Si pensamos en los estados como colores, en un autómatata con dos estados cada celda puede ser, por ejemplo, blanca o negra.

Las celdas pueden cambiar de estado. Las *reglas* determinan cómo cambian los estados.

El cambio de estado opera así: cada celda *mira* a su alrededor y recopila información del vecindario (el vecindario depende de la naturaleza del AC particular).

Basado en su propio estado, en sus estados vecinos, y en las reglas del AC, la celda adoptará un nuevo estado. Todas las celdas cambian al mismo tiempo.

JUEGO DE LA VIDA DE CONWAY

En cada paso puede ocurrir:

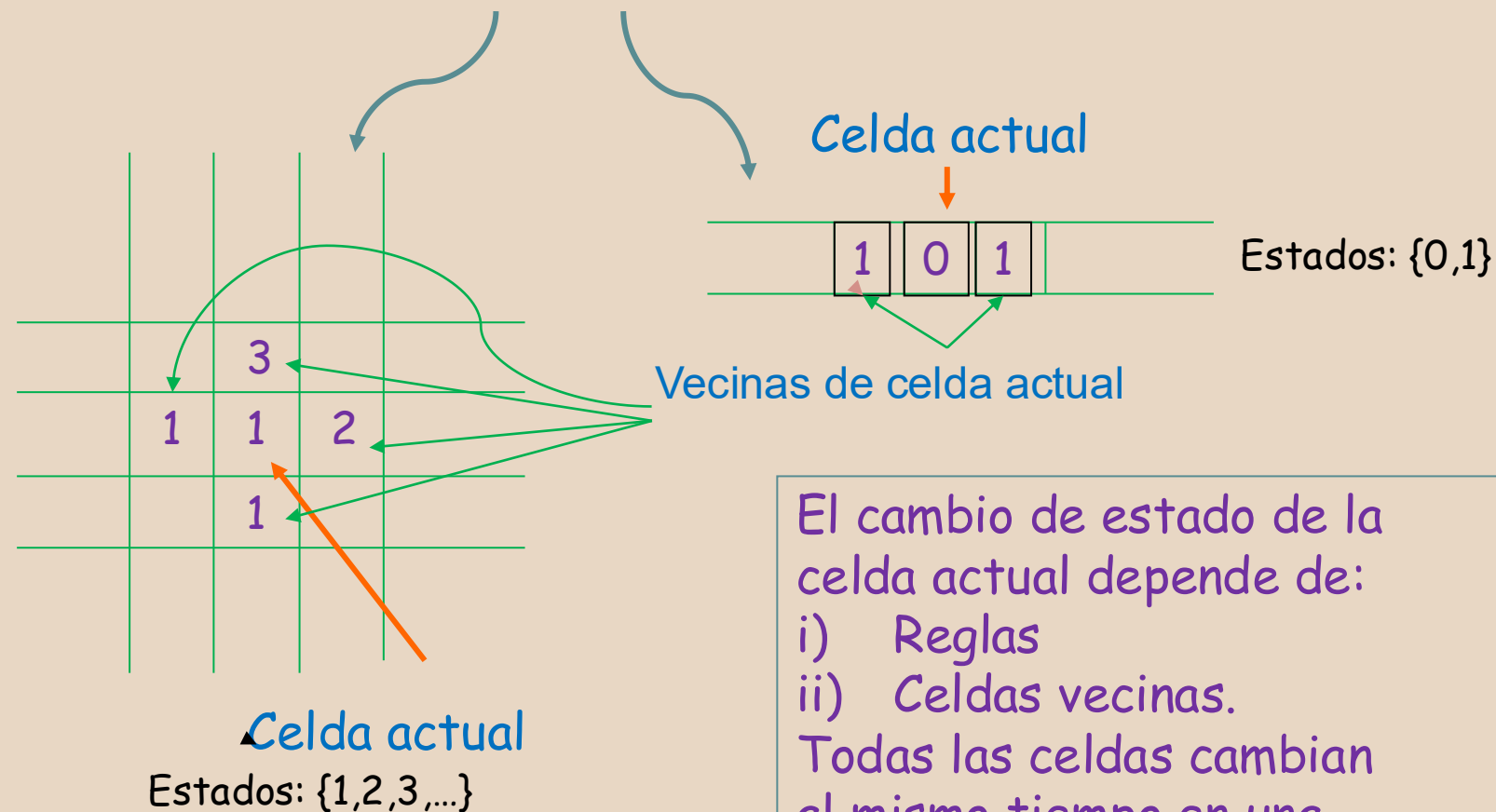
- 1.Cualquier celda viva, con menos de dos vecinos, muere (de soledad).**
- 2.Cualquier celda viva con más de tres vecinos, muere (por sobrepoblación).**
- 3.Cualquier celda viva, con dos o tres vecinos vivos, no cambia su estado.**
- 4.Cualquier celda muerta con exactamente tres vecinos vivos, vuelve a la vida.**

- El patrón inicial constituye la primera generación del sistema. La segunda generación es creada aplicando simultáneamente las reglas (que son las 1, 2, 3 y 4) a cada celda de la primera generación. Los nacimientos y muertes ocurren al mismo tiempo. Las reglas se aplican repetidamente para la creación de las generaciones futuras.

APLICACIONES DE LOS AC

- Simulación de procesos biológicos
- Simulación del crecimiento de células cancerígenas
- Modelos depredador – presa
- Arte
- Simulación de incendios forestales
- Simulación de movimientos sociales
- ...muchos más. Es un campo de investigación muy activo.

División del espacio problema en celdas o células



VISIÓN GENERAL

- Los autómatas celulares se pueden usar para modelar sistemas complejos con reglas simples.
- Aspectos clave:
 - División del espacio problema en **celdas**.
 - Cada celda puede estar en uno de varios **estados** finitos.
 - Las celdas son afectadas por sus **vecinos** de acuerdo a reglas.
 - Todas las celdas son afectadas **simultáneamente** en una generación.
 - Las reglas se re-aplican durante varias **generaciones**.

AUTÓMATA CELULAR

- Un AC es un retículo espacial de N celdas, cada una de las cuales representa un estado k en el tiempo t .
- Cada celda sigue la(s) misma(s) regla(s) simple(s) para actualizar su estado.
- El estado de la celda s en el tiempo $t+1$ depende de su propio estado y de los estados de un cierto número de celdas vecinas en el tiempo t .

AUTÓMATA CELULAR (CONT.)

- Para AC **unidimensionales**, la vecindad de una celda consiste de la celda misma y r vecinos a cada lado. De aquí, k (el estado de la celda actual) y r son los parámetros del AC.
- Los AC son **sistemas dinámicos discretos** que tienen la capacidad de describir **sistemas dinámicos continuos**.

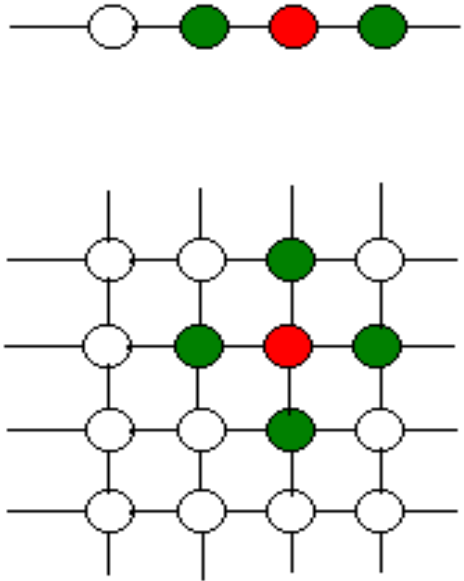
AUTÓMATA CELULAR (CONT.)

- ACs también han sido usados para simulación de:
 - Fluidos dinámicos.
 - Oscilaciones químicas.
 - Crecimiento de cristales.
 - Formación de galaxias.
 - Patrones fractales en conchas de moluscos.
 - Procesamiento de imágenes.
 - ...

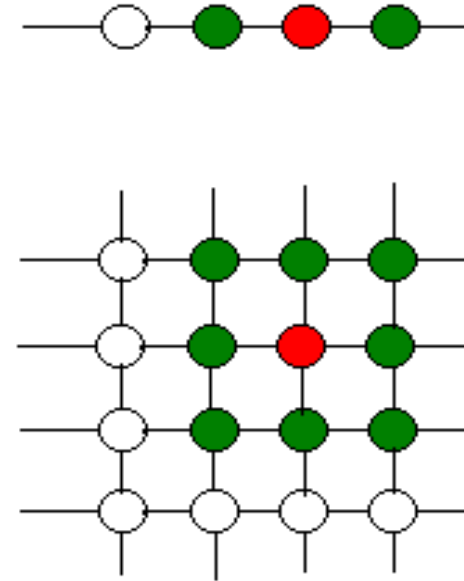
AUTÓMATA CELULAR

- Tipos de vecindad:

Von Neumann



Moore



CLASES DE AUTÓMATAS CELULARES (WOLFRAM)

- **Clase 1:** Después de un número finito de pasos, el AC tiende a alcanzar un **estado único**, a partir de casi todos los posibles estados iniciales (puntos límite).
- **Clase 2:** El AC crea patrones que se **repiten periódicamente** o son estables (ciclos límite) – probablemente equivalente a un autómata finito para el reconocimiento de una gramática regular.

CLASES DE AUTÓMATA CELULAR (WOLFRAM)

- **Clase 3:** El AC lleva a patrones **aperiódicos caóticos**, donde las propiedades estadísticas de estos patrones son casi idénticas (después de un período de tiempo suficientemente grande) a los patrones iniciales.
- **Clase 4:** Después de un número finito de pasos, el AC normalmente **muere**, pero hay unos pocos patrones estables (periódicos) posibles (por ejemplo, el juego de la vida) – Se cree que estos AC son capaces de realizar una computación universal.

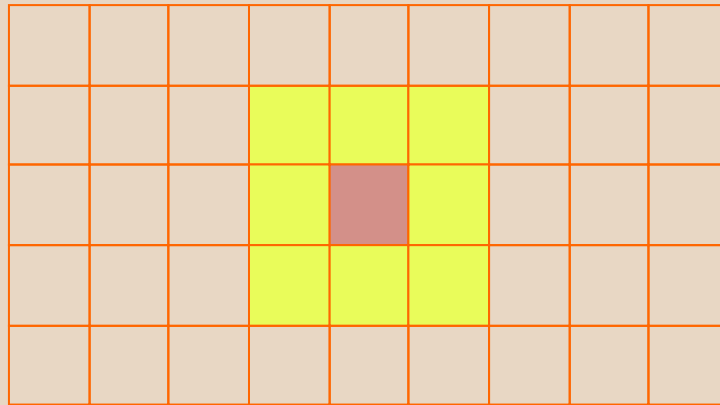
EXITO Y FRACASO DE MODELOS COMPLEJOS

Nuevos paradigmas:

- Las investigaciones muestran que la complejidad es un pre-requisito para el éxito de los sistemas vivos (Jacobs, Langton, Wolfram, Kaufmann, Fotheringham, Longley & Batty, Frankhauser & Sadler).
- Lecciones aprendidas:
 - *Complejidad $\neq$ complicación*

AUTÓMATA 2-DIMENSIONAL

Un autómata celular 2-dimensional consiste de una grilla infinita (o finita) de celdas, cada una de ellas en un estado (de un conjunto finito de estados). El tiempo es discreto y el estado de una celda en el tiempo t es función de los estados de sus vecinos en el tiempo $t-1$.



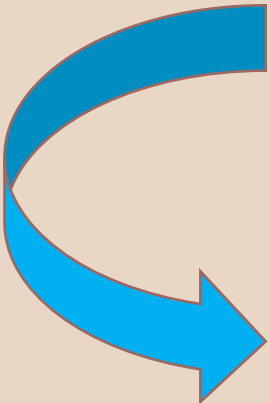
UN EJEMPLO ELEMENTAL

0	0	0	0	1	1	1	0	0	0	0
---	---	---	---	---	---	---	---	---	---	---

Si $A_i = 0$ y $A_{i-1} = 1$ entonces $A_i = 1$

Si $A_i = 1$ y $A_{i-1} = 0$ entonces $A_i = 0$

UN EJEMPLO ELEMENTAL (CONT.)



0	0	0	0	1	1	1	0	0	0	0
---	---	---	---	---	---	---	---	---	---	---

Estado inicial

Si $A_i = 0$ y $A_{i-1} = 1$ entonces $A_i = 1$

Si $A_i = 1$ y $A_{i-1} = 0$ entonces $A_i = 0$

0	0	0	0	0	1	1	1	0	0	0
---	---	---	---	---	---	---	---	---	---	---

Estado final

UN EJEMPLO ELEMENTAL (CONT.)

0	0	0	0	0	0	1	1	1	0	0
---	---	---	---	---	---	---	---	---	---	---

UN EJEMPLO ELEMENTAL (CONT.)

0	0	0	0	0	0	0	0	1	1	1	0
---	---	---	---	---	---	---	---	---	---	---	---

UN EJEMPLO ELEMENTAL (CONT.)

0	0	0	0	0	0	0	0	1	1	1	0
0	0	0	0	0	0	0	0	1	1	1	0

UN EJEMPLO ELEMENTAL (CONT.)

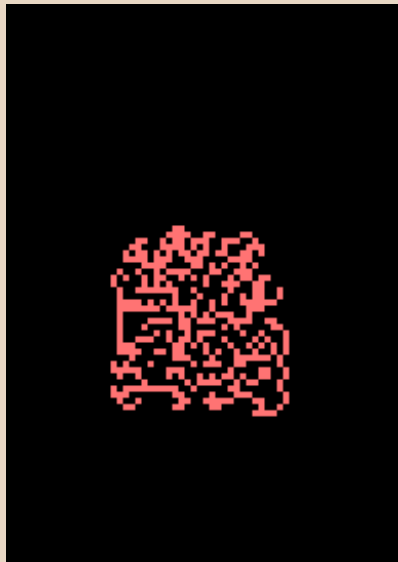
[illegible]

UNA MIRADA A LAS HORMIGAS VIRTUALES DE LANGTON

1. Comienzan en un punto arbitrario en un plano euclideo.
2. Avanzan un paso:
 - i. Si la hormiga está parada en una celda blanca, la pinta rosada, y avanza girando 90 grados a la derecha.
 - ii. Si la hormiga está en una celda rosada, la pinta blanca y avanza girando 90 grados a la izquierda.
3. Ir al paso 2 y repetir durante un número arbitrario de iteraciones.

HORMIGA VIRTUAL DESPUÉS DE MUCHAS ITERACIONES

- Aunque el movimiento de las hormigas es determinístico, las primeras 10.000 iteraciones dan la impresión de ser caóticas e impredecibles.
- Eventualmente, sin embargo, las hormigas comienzan a formar un camino.



5,000 it.



10,000 it.



12,000 it.

OTRO EJEMPLO: INCENDIOS FORESTALES

Hay que definir un número *adecuado* de estados. Tiene sentido para una celda estar **vacía**, **árbol_ok**, o **árbol_fuego**.

Significado:

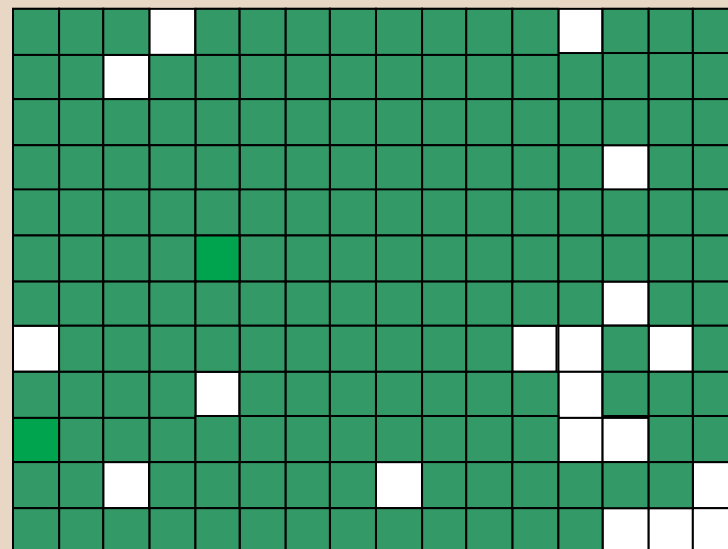
vacía: no hay árbol

árbol_ok: aquí hay un árbol, y está saludable

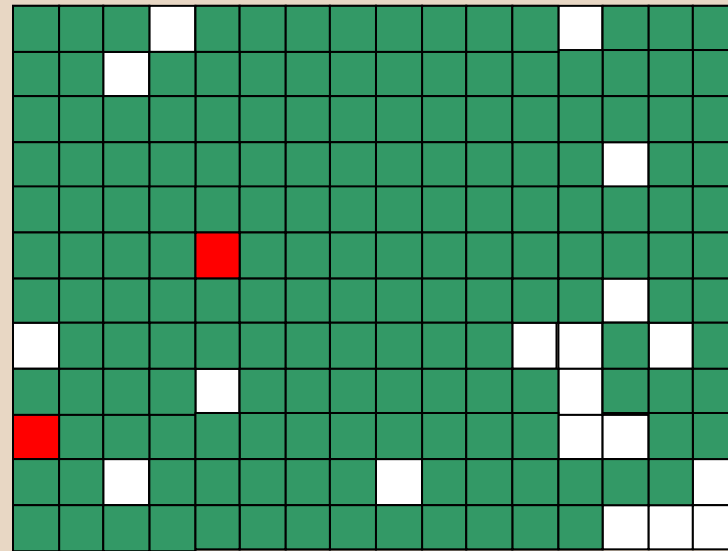
árbol_fuego: aquí hay un árbol, y está en llamas

Cuando se visualiza un AC, es habitual usar colores para representar los estados. Elegimos blanco, verde y rojo.

Un bosque denso



Un bosque denso con un par de árboles en llamas...



LOS ÁRBOLES ...

Definimos la vecindad. En ejecución, las celdas cambian su estado bajo la influencia de sus vecinos. Hay que decidir cuáles son los *vecinos*.

Es habitual considerar la vecindad de Moore (8 vecinos).

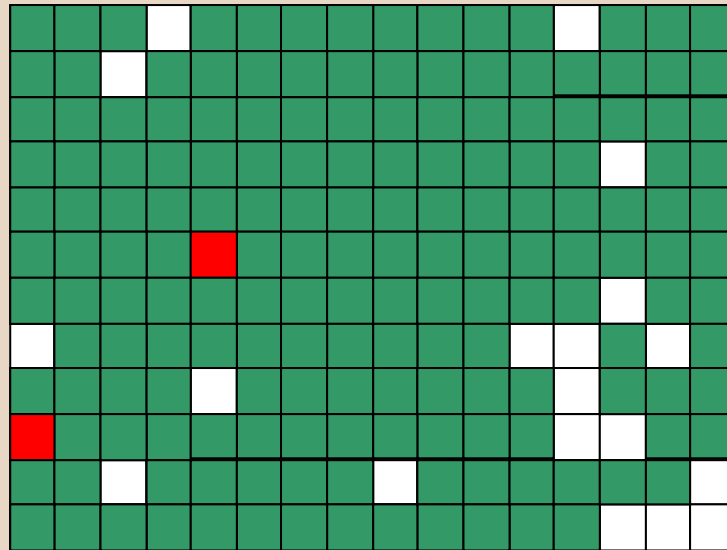
L A S R E G L A S

Importante: ¿Cómo actualizamos los estados para el próximo paso?
Usamos reglas sensibles.

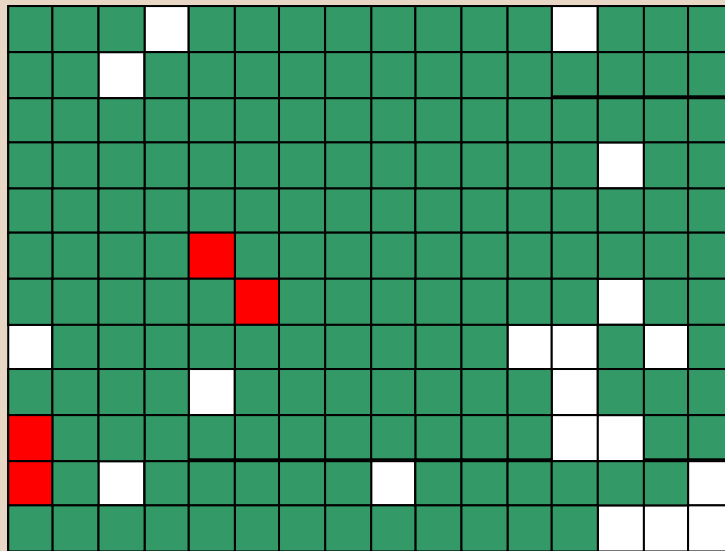
Ejemplo:

- Si un árbol no está en llamas, y tiene n vecinos en llamas, estará en llamas en el próximo paso con probabilidad $n/8$.
- Si un árbol se mantiene en llamas durante tres pasos, muere.

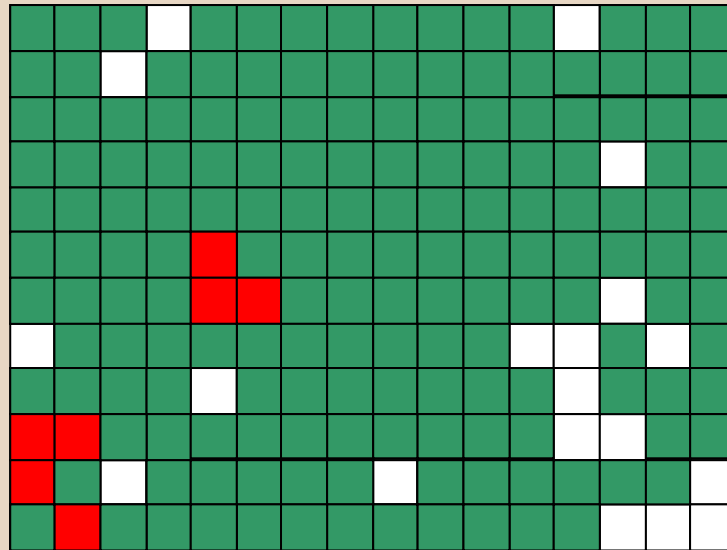
Paso 0



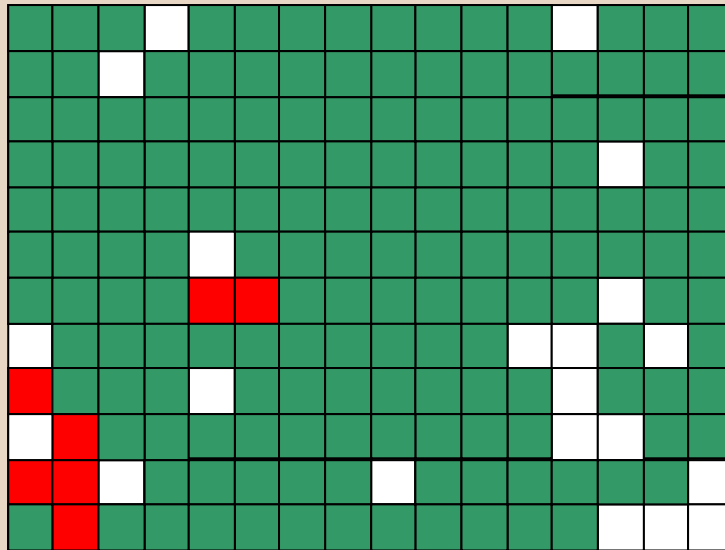
Paso 1



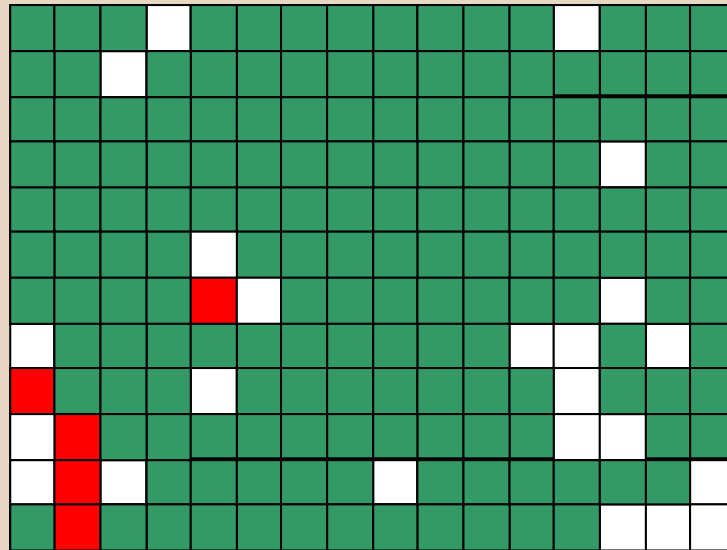
Paso 2



Paso 3



Paso 4... y así sucesivamente



UNA MIRADA FORMAL

Teoría de Autómatas es una rama de Ciencias de Computación que:

- a) Trata de responder preguntas como “¿Qué pueden hacer los computadores, y qué es lo que está más allá de sus capacidades?”.
- b) Ayuda a crear y estudiar nuevos modelos de computación de forma no ambigua.
- c) Tiene muchas implicaciones prácticas y es la base para muchas aplicaciones del mundo real.

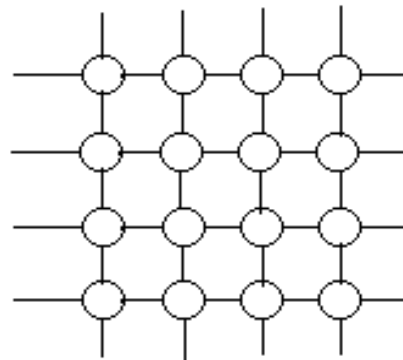
AUTOMATA CELULAR

- Un AC es un modelo de un computador paralelo.
- Un AC consiste de procesadores (celdas), conectadas normalmente en una grilla n-dimensional.

1-D CA



2-D CA



AUTOMATA CELULAR

- Un componente importante de un AC es su grafo de interconexión, G , el cual es, típicamente, una matriz n -dimensional.
- Cada celda del AC puede estar en uno de varios estados posibles. El conjunto de estados, Q , de un AC es el conjunto de todos los estados posibles en los cuales puede estar una celda.
- El par (G, Q) es normalmente conocido como Espacio de Celdas del AC.

AUTÓMATA CELULAR

- Una configuración x , de un AC es un mapeamiento desde un grafo Γ al conjunto de estados, el cual asigna un estado desde el conjunto de estados Q a cada nodo en el grafo Γ , i.e.

$$x: \Gamma \rightarrow Q$$

$$x(i) = q, \text{ donde } i \in \Gamma \text{ y } q \in Q$$

- Una configuración de un AC describe el estado completo del AC en una escala global.

AUTÓMATA CELULAR

- La computación de ACs, es **un proceso local**. El próximo estado de cada celda depende sólo de su estado actual, y de los estados de sus vecinos más próximos.
- Así, necesitamos definir el concepto de **vecindad de celdas**.
- La vecindad de una celda en un AC, es la colección de celdas situadas a una “distancia” **r** o menor desde la celda en cuestión.

AUTÓMATA CELULAR

- Cada celda de un AC es una Máquina de Estado Finito simple.
- La **dinámica local** (función de transición) de una celda, denotada por δ , es una función, que recibe como entrada el estado de una celda y de sus “vecinos”, y computa el próximo estado de la celda.
- Por ejemplo, la dinámica local de un AC de una dimensión se puede definir como:

$$\delta(x_{i-1}, x_i, x_{i+1}) = x_i'$$

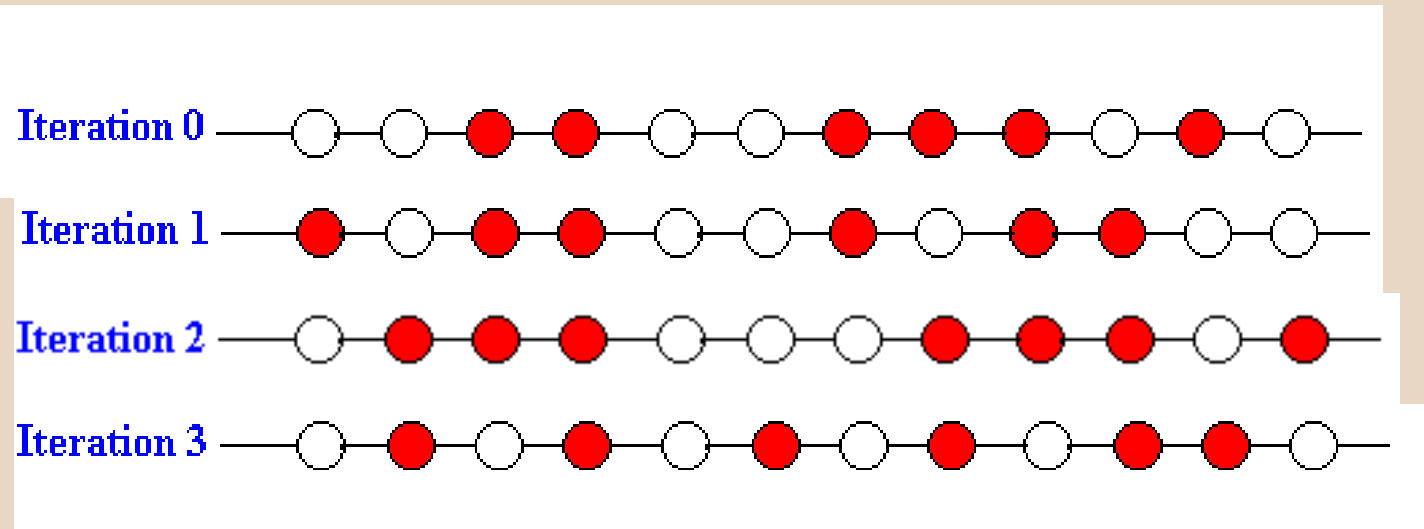
AUTÓMATA CELULAR

- La dinámica local se expresa a menudo como una tabla:

x_{i-1}, x_i, x_{i+1}	000	001	010	011	100	101	110	111
$\delta(x_{i-1}, x_i, x_{i+1})$	1	0	0	1	0	1	1	0

AUTÓMATA CELULAR

- Ejemplo de evolución de un AC de 1 dimensión basado en la dinámica local mostrada:



○ - "0"
● - "1"

AUTÓMATA CELULAR

- Formalmente, un **Autómata Celular** es una cuádrupla $M = (\Gamma, Q, N, \delta)$, donde:
 - Γ - grafo de interconexión,
 - Q - conjunto de estados,
 - N - vecindad (por ejemplo, von Neumann, etc.),
 - δ - dinámica local.

AUTÓMATA CELULAR

- La dinámica local, δ , de un AC describe la computación que ocurre localmente en cada celda.
- La computation global del AC como un sistema es capturada por la noción de dinámica global, T .

AUTÓMATA CELULAR

- La dinámica global, T , de un AC es un mapeamiento desde el conjunto de configuraciones C a sí mismo, i.e.

$$T: C \rightarrow C$$

- Luego, La dinámica global describe cómo el estado completo del AC cambia desde una instancia a la siguiente.

AUTÓMATA CELULAR

- Ya que la computación global está determinada por la computación de cada celda individual, la dinámica global, T , se define en función de la dinámica local, δ :

$$T(\mathbf{x})_i := \delta(x_{i-1}, x_i, x_{i+1})$$

AUTÓMATA CELULAR

- A partir de una configuración inicial, x , el AC evoluciona en el tiempo computando las iteraciones sucesivas de la dinámica global:

$$x, T(x), T^2(x)=T(T(x)), \dots, T^n(x), \dots$$

- Luego, podemos ver la evolución en el tiempo de un AC como la computación de una órbita de un sistema dinámico discreto.

EJEMPLO SIMPLE

Ejemplo simple de un autómata celular en Python.

Este ejemplo utiliza un autómata celular unidimensional conocido como "regla 30".

EJEMPLO SIMPLE

Se puede experimentar con diferentes reglas cambiando el valor de la variable **regla**.

Además, se puede ajustar el número de generaciones y la longitud de la línea celular según sus preferencias.

CÓDIGO

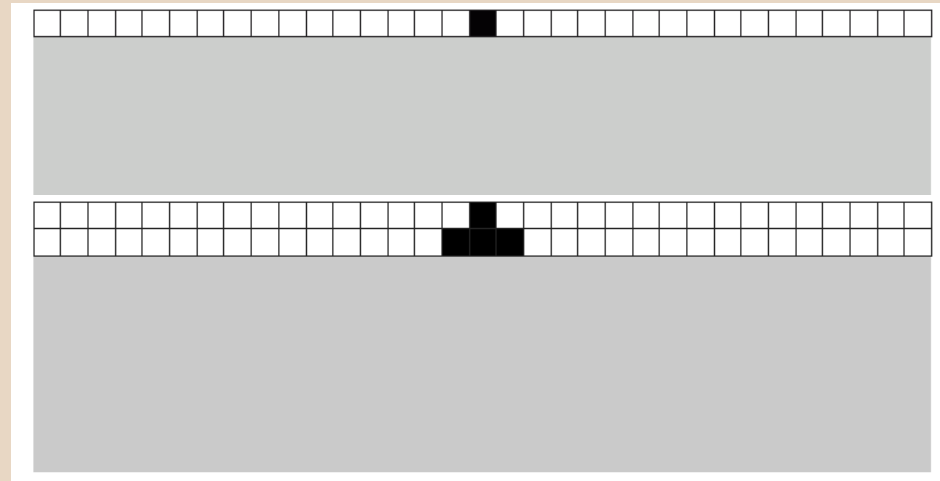
```
• def regla30(a, b, c):  
•     """  
•     Regla 30 para la evolución de un autómata celular.  
•     """  
•     return a ^ (b or c)  
•  
•     def automata_celular(regla, generaciones, longitud):  
•         """  
•         Simula la evolución de un autómata celular.  
•  
•         :param regla: Regla de transición del autómata celular.  
•         :param generaciones: Número de generaciones a simular.  
•         :param longitud: Longitud de la línea celular.  
•         """
```

- `# Inicializamos la línea celular con un solo "1" en el centro.`
- `linea_celular = [0] * longitud`
- `linea_celular[longitud // 2] = 1`
- `# Imprimimos la línea inicial.`
- `print("Generación 0:", "".join(map(str, linea_celular)))`
- `# Simulamos las generaciones sucesivas del autómata celular.`
- `for generacion in range(1, generaciones + 1):`
- `nueva_linea = [0] * longitud`

- `# Aplicamos la regla para cada célula en la línea.`
- `for i in range(longitud):`
- `izquierda = linea_celular[i - 1] if i > 0 else 0`
- `actual = linea_celular[i]`
- `derecha = linea_celular[i + 1] if i < longitud - 1 else 0`
- `nueva_linea[i] = regla(izquierda, actual, derecha)`
- `# Actualizamos la línea celular para la siguiente generación.`
- `linea_celular = nueva_linea`
- `# Imprimimos la línea actual.`
- `print(f"Generación {generacion}:", "".join(map(str, linea_celular)))`
- `# Configuración del autómata celular.`
- `regla = regla30`
- `generaciones = 10`
- `longitud = 21`
- `# Simulamos la evolución del autómata celular.`
- `automata_celular(regla, generaciones, longitud)`

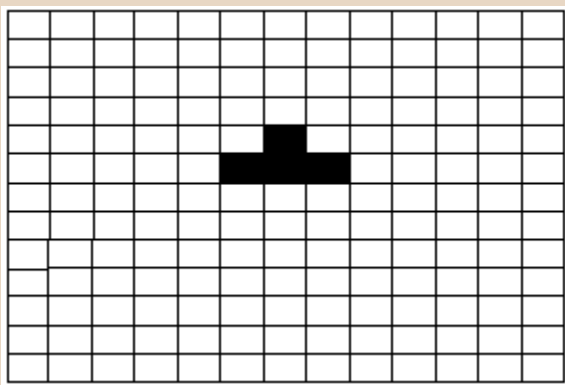
AUTÓMATA CELULAR SIMPLE

- Para un conjunto dado de reglas, y una configuración inicial dada, la evolución determinística del AC se puede ver a continuación.

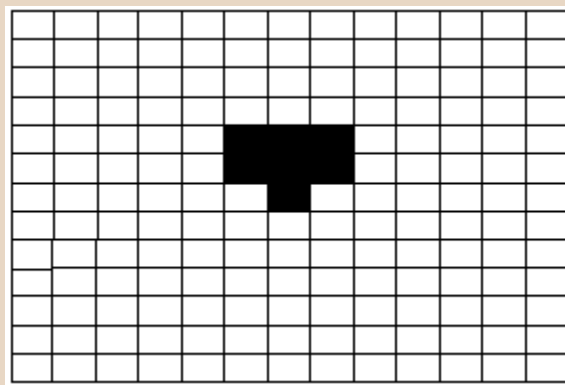


Reforzando las ideas previas... Y otro ejemplo

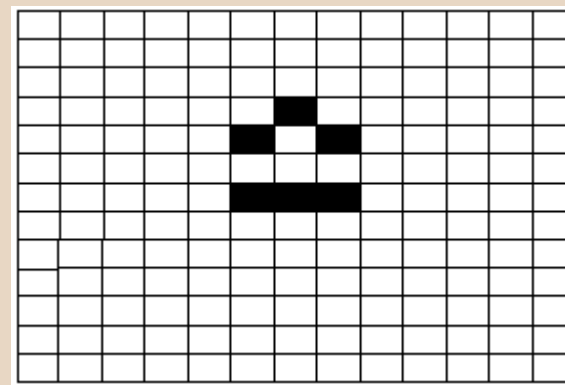
- En la configuración inicial del autómata, a cada celda se le asigna un valor *de partida*, dentro de un rango de valores posible.
- Por ejemplo, si el rango de valores es 0 o 1, cada celda podría asumir el valor 0 o 1 en la configuración inicial.
- En general se usan colores para la representación visual en un computador.
- Cada celda está asociada a una regla de transición.



Inicial



Paso 1



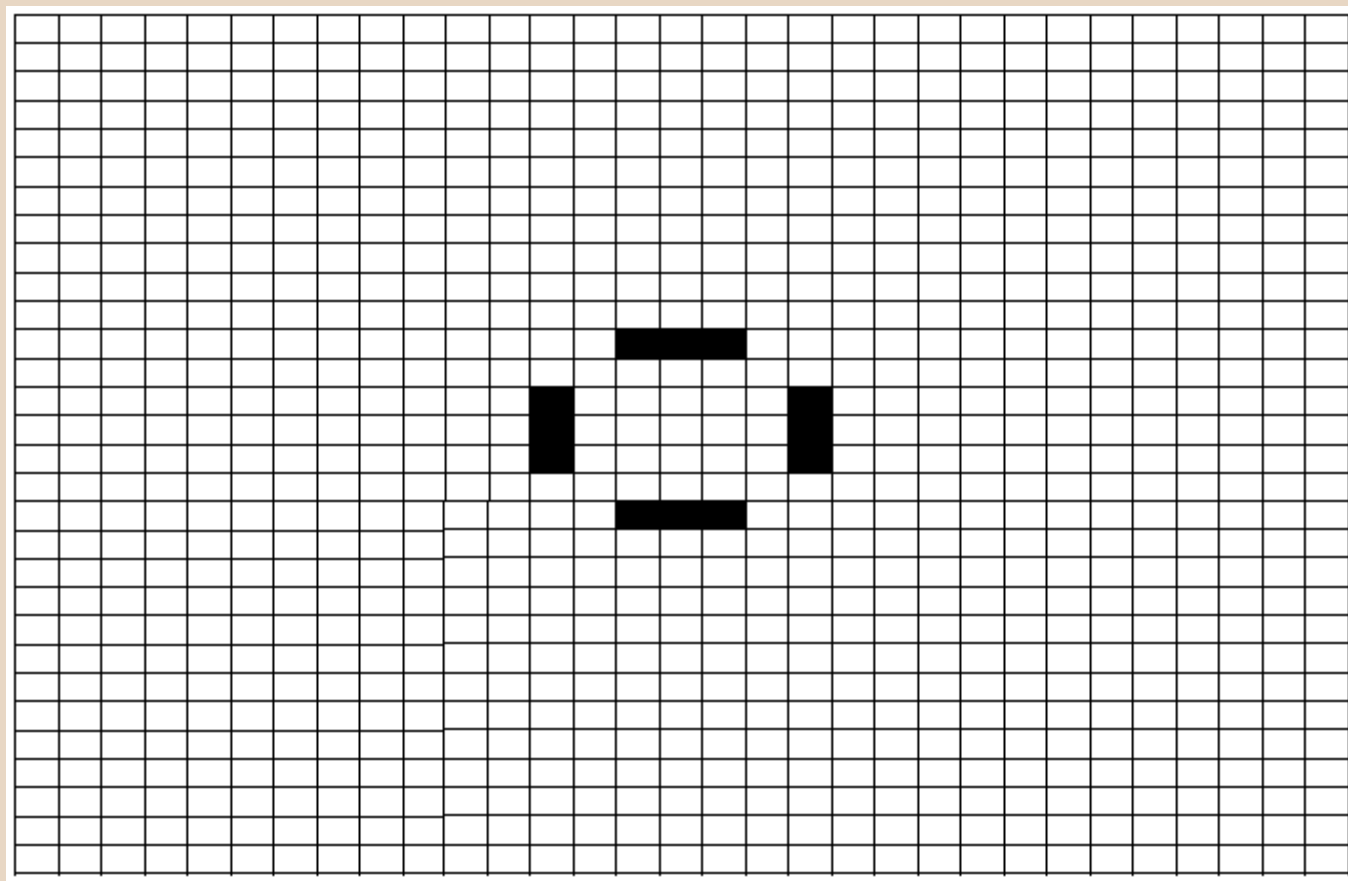
Paso 2

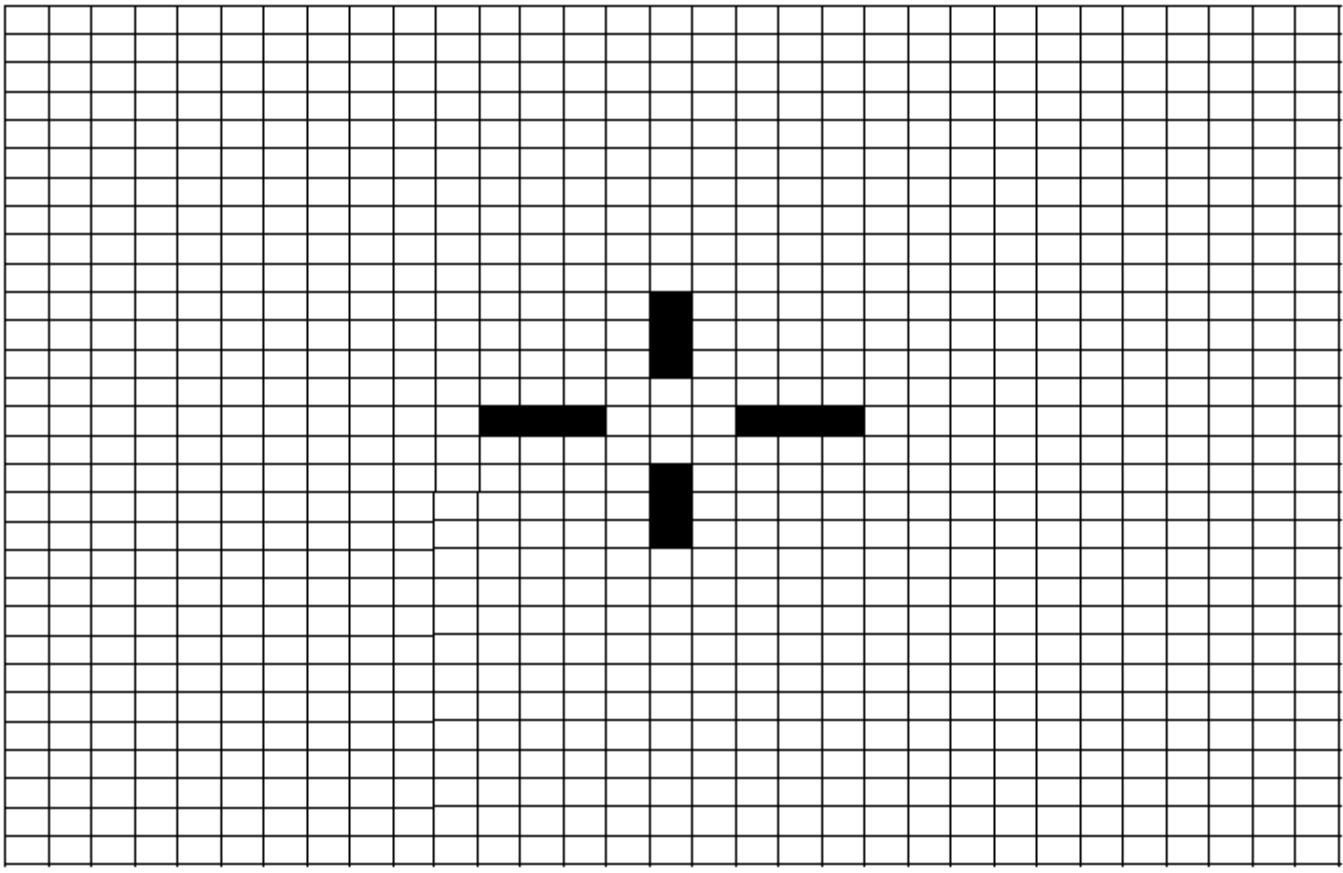
En este ejemplo, la configuración inicial para todas las celdas es el estado 0, excepto para cuatro celdas que están en estado 1

•Las reglas de transición son:

- Una celda permanece en el estado 1 (negra), si tiene dos o tres vecinos en estado 1.
- Una celda cambia a negra, si tiene exactamente tres vecinas negras.
- En caso contrario, cambia a blanca.

Veamos cómo funciona





En este ejemplo:

- Cada celda tiene 8 vecinos,
- Cada celda puede estar en uno de los valores posibles en un instante dado,
 - La transición ocurre en tiempos discretos
 - esto ayuda a imaginar la transición de todas las celdas
- Cada estado se codifica como un color único,
 - La regla de transición toma como entrada el estado actual (i.e., los valores actuales) de todas las celdas en una vecindad dada, y genera el próximo estado (i.e., el próximo valor) de una celda dada.
 - Cuando se aplica a todas las celdas, individualmente, en un autómata celular, **el próximo estado del autómata celular (completo) se genera a partir del estado actual.**
 - Luego, el estado siguiente del AC se **copia** al estado actual (nuevo) y **se repite el proceso** tantas veces como se quiera.

SISTEMAS DINÁMICOS

- A veces ciertos puntos en el dominio de algunos sistemas dinámicos exhiben propiedades muy interesantes:
 - Un punto x , en el dominio de F se llama **atractor** ssi hay una vecindad de x tal que cualquier punto en esa vecindad, bajo iteración de F , tiende a aproximarse a x
 - Un punto x , en el dominio de F se llama **repelente** ssi existe una vecindad de x tal que cualquier punto en esa vecindad, bajo iteración de F , tiende a separarse de x

SISTEMAS DINÁMICOS

- Los objetivos de los estudios de sistemas dinámicos son:
 - a) Predecir el comportamiento asintótico de largo plazo de un sistema, dado algún punto inicial x .
 - b) Identificar puntos interesantes en el dominio del sistema, tales como atractores, repelentes, puntos periódicos, etc.

SISTEMAS DINÁMICOS

- Para algunos sistemas dinámicos simples, la predicción del comportamiento asintótico de largo plazo es muy simple.
- Para otros sistemas, no se puede predecir más que unas pocas iteraciones hacia el futuro. Tales sistemas impredecibles se conocen normalmente como **sistemas caóticos**.

DINÁMICA CAÓTICA

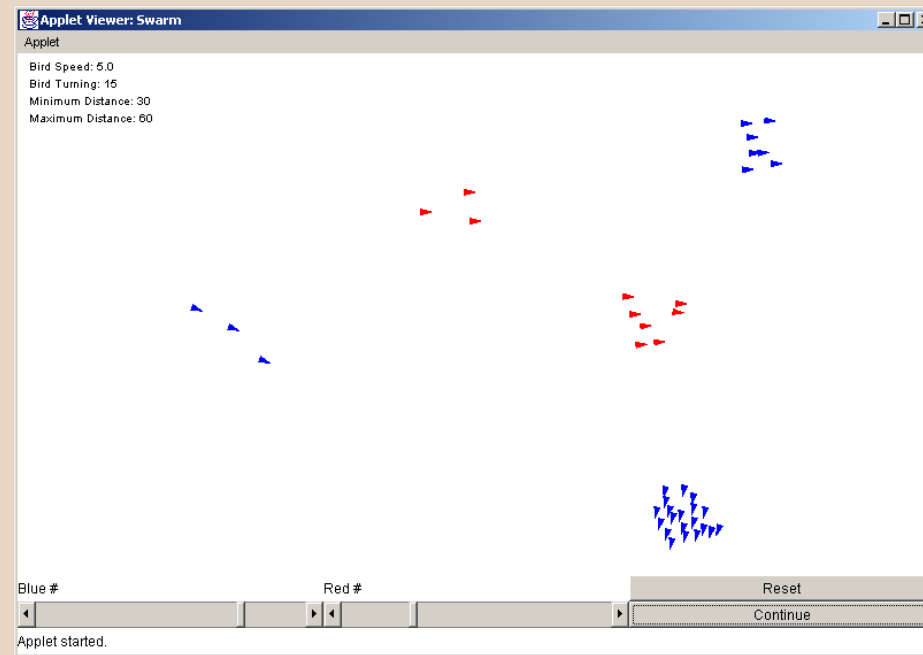
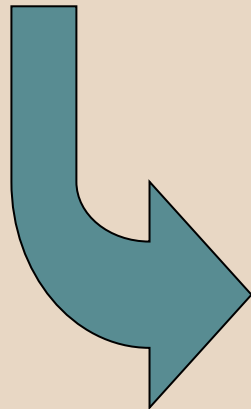
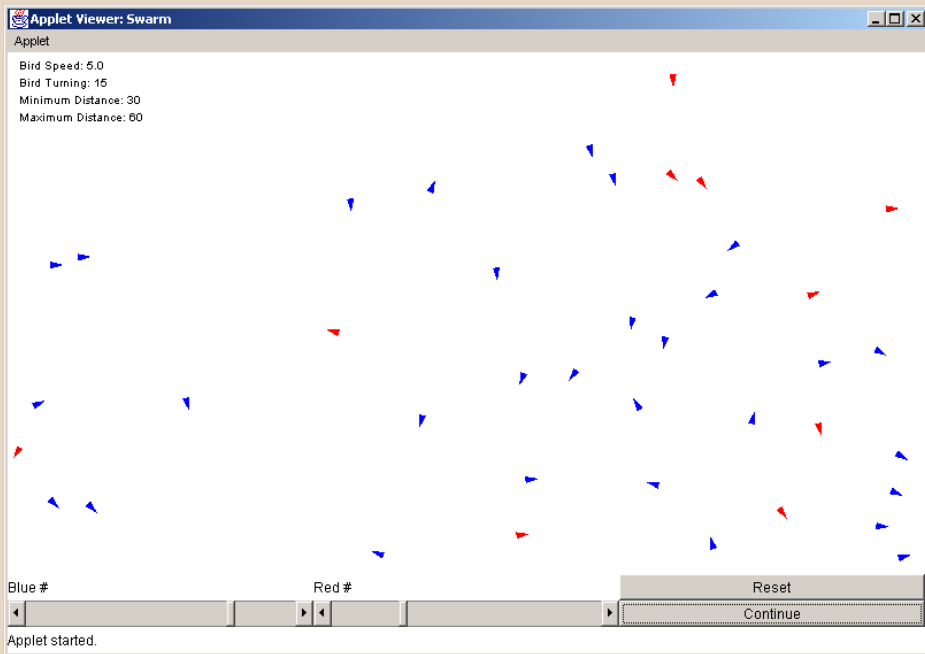
- Un sistema dinámico caótico tiene 3 características distinguidas:
 - a) **Transitividad topológica** - esto implica que el sistema no puede ser descompuesto y estudiado pieza por pieza.
 - b) **Dependencia sensible a las condiciones Iniciales** - esto implica que las simulaciones numéricas son inútiles, ya que errores pequeños se magnifican bajo iteración, y por lo tanto las órbitas que obtenemos de la computación no tienen nada que ver con las órbitas reales del sistema.
 - c) **El conjunto de puntos periódicos es denso en el dominio del sistema** - introduce impredecibilidad, un elemento de regularidad.

AUTÓMATAS CELULARES COMO SISTEMAS DINÁMICOS

- El comportamiento de un AC en términos de iteración de su dinámica global, T , puede ser considerado un sistema dinámico.
- Dependiendo de la configuración inicial y la elección de la dinámica local, δ , el AC puede mostrar cualquier comportamiento típico de un sistema dinámico - fijo, periódico, o incluso caótico.
- Ya que ACs pueden modelar con precisión numerosos fenómenos y sistemas del mundo real, la comprensión del comportamiento de un AC nos puede llevar a una mejor comprensión del mundo que nos rodea.

COMPORTAMIENTO EMERGENTE

Hay un modelo Famoso: BOIDS



B O I D S (B A N D A D A)

Cada miembro tiene acceso a todo el espacio, pero reacciona sólo ante otros miembros que estén dentro de un radio específico.

El modelo básico consiste de tres simples comportamientos:

1. **Separación**: se mueven evitando chocar con otros miembros.
2. **Alineación**: se mueven hacia el miembro más cercano que avanza en el grupo.
3. **Cohesión**: se dirigen a la posición promedio de los miembros locales

B O I D S (C O N T .)

Modelos más elaborados incluyen predecir y evitar obstáculos y la existencia de metas.

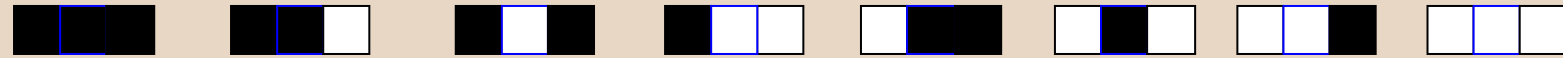
Evitar obstáculos permite el vuelo a través de ambientes simulados donde se le “hace el quite” a objetos estáticos.

Se pueden usar en aplicaciones de animación computacional...

Por ejemplo, en Batman Returns, con bandadas de murciélagos

AUTÓMATA CELULAR ELEMENTAL

Pensemos que cada célula tiene un vecino a la izquierda y un vecino a la derecha. Así cada celda con sus vecinos puede tener una de las siguientes configuraciones:



Reemplace cada celda negra con un 1 y cada celda blanca con un 0

111	110	101	100	011	010	001	000
-----	-----	-----	-----	-----	-----	-----	-----

111	110	101	100	011	010	001	000
■	■	■	■	■	■	■	■

Cada celda amarilla puede ser llenada con un 0 o con un 1, lo que da un total de $2^8 = 256$ reglas de actualización posibles.

Ejemplo: Considere que el string **0 1 1 0 1 0 1 0** pueda usarse para representar la regla de actualización

1 1 1	1 1 0	1 0 1	1 0 0	0 1 1	0 1 0	0 0 1	0 0 0
0	1	1	0	1	0	1	0

O de forma equivalente



Ahora pensemos en 01101010 como la expansion binaria del número:

$$0 \cdot 2^7 + 1 \cdot 2^6 + 1 \cdot 2^5 + 0 \cdot 2^4 + 1 \cdot 2^3 + 0 \cdot 2^2 + 1 \cdot 2^1 + 0 \cdot 2^0 = 64 + 32 + 8 + 2 = 106.$$

Por lo que esta regla de actualización es conocida como la regla **# 106**

Regla #45=32+8+4+1

$$= 0 \cdot 2^7 + 0 \cdot 2^6 + 1 \cdot 2^5 + 0 \cdot 2^4 + 1 \cdot 2^3 + 1 \cdot 2^2 + 0 \cdot 2^1 + 1 \cdot 2^0 = 00101101$$

Regla #30=16+8+4+2

$$= 0 \cdot 2^7 + 0 \cdot 2^6 + 0 \cdot 2^5 + 1 \cdot 2^4 + 1 \cdot 2^3 + 1 \cdot 2^2 + 1 \cdot 2^1 + 0 \cdot 2^0 = 00011110$$

La convención para denominar de esta forma a las 256 reglas de actualización diferentes se debe a Stephen Wolfram. Uno de los pioneros de los automata celulares (autor del libro *A New Kind of Science*). Wolfram argumenta que los AC no son un hecho aislado, sino un paradigma para todos los modelos científicos.

Autómata generado usando la Regla 30

(aparece en la naturaleza...)

1	1	1	1	1	0	1	0	1	1	0	0	1	1	0	0	0	0
0	0	0	1	1	1	1	1	1	0								



REGLA 184

- **Regla 184** es un AC notable por su capacidad de resolver una cantidad de fenómenos aparentemente diferentes:
- Se puede usar para modelar un flujo de tráfico en un solo carril, por ejemplo en una autopista (que es la base para sistemas de simulación de tráfico más complejos).
- En este modelo, partículas (que representan vehículos) se mueven en una sola dirección, deteniéndose y avanzando dependiendo de los vehículos delante de ellos. El número de partículas permanece constante durante la simulación. Por esa razón a esta regla se la conoce también como **regla del tráfico**.

FLUJO DEL TRÁFICO

Cada celda en estado 1 corresponde a un vehículo, y cada vehículo se mueve hacia Adelante si y solamente si hay un espacio delante de él.



Aunque primitiva, la regla 184 modela las características emergentes del tráfico real: vehículos agrupados que se mueven libremente separados por espacios cuando el tráfico es liviano, y oleadas de tráfico con detenciones y arranques cuando el tráfico es pesado.

MODELO DE PREDADOR - PRESA

¿Recuerdan el ejemplo de zorros y conejos?

MODELO LOTKA - VOLTERRA

El modelo Lotka–Volterra se usa frecuentemente para describir la dinámica de sistemas en la cual interactúan dos especies: el depredador y la presa.

El modelo se simplifica con algunas suposiciones: (1) solo existen dos especies: zorros y conejos; (2) los conejos nacen y mueren a causa de los depredadores o muerte normal; (3) los zorros nacen y su tasa de natalidad es afectada por la tasa de captura, y mueren de forma natural (de viejos).

LOTKA - VOLTERRA

La característica de este modelo es que los cambios en la población, tanto del depredador como de la presa, se explican en términos de la otra población:

El tamaño de la población de zorros, tiene un efecto negativo en la población de conejos, y el tamaño de la población de conejos tiene un efecto positivo en la población de zorros.

LOTKA - VOLTERRA

El modelo se describe con este par de ecuaciones diferenciales de primer orden:

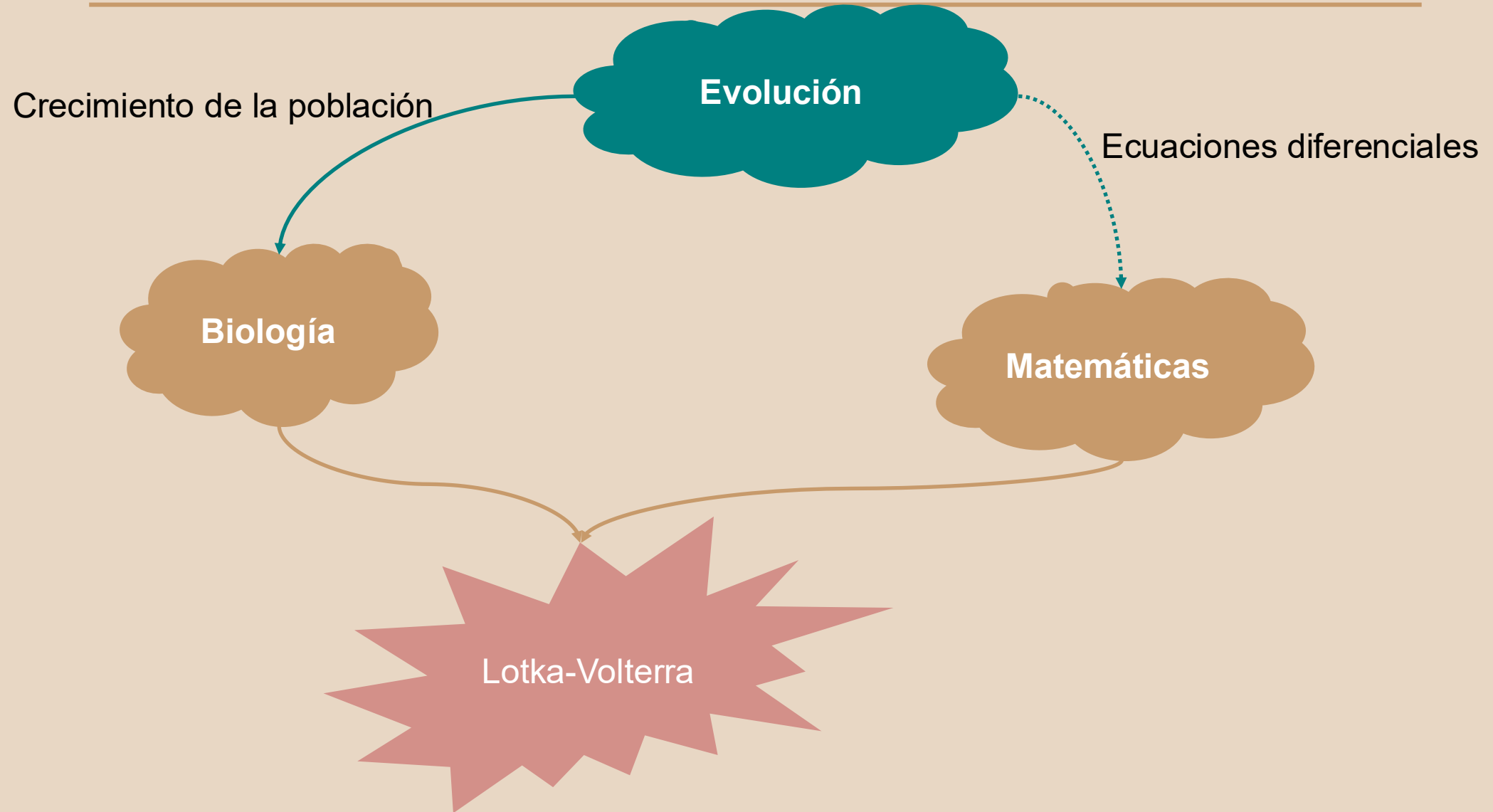
$$dC/dt = \text{Tasa de nacimiento de conejos} * C - \text{tasa de muerte de conejos} * C * Z$$

$$dZ/dt = \text{Tasa de nacimiento de zorros} * R * Z - \text{tasa de muerte de zorros} * Z$$

Depredador-Presa – Aspectos básicos

- Aplicable a otros problemas:
 - herbívoros - plantas
 - parásitos - anfitriones
- Oscilaciones en el tamaño de la población tanto de depredadores como de presas.
- Peak mínimo de los depredadores oscila ligeramente bajo el peak mínimo de la presa.

Depredador-Presa – Historia



Depredador-Presa – Suposiciones

- Si no hay depredador, la población crecerá exponencialmente.
- La población del depredador morirá de hambre si no hay presas.
- Los depredadores pueden consumir cantidades infinitas de presas.
- Las poblaciones se mueven randómicamente a través del ambiente.

Depredador-Presa – Historia

- Alfred Lotka (norteamericano) y Vito Volterra (italiano).
- Volterra quería modelar las poblaciones de peces en el mar Adriático.
- Publicó sus trabajos en Nature:

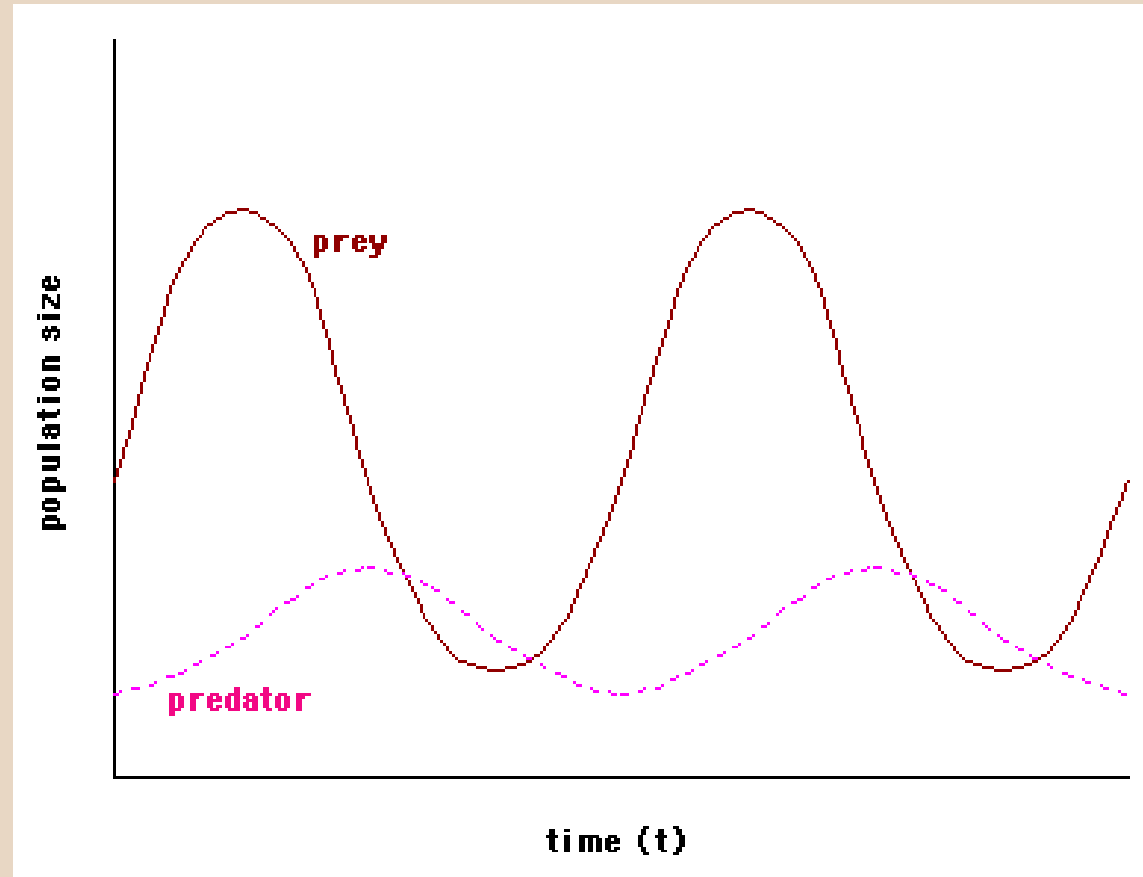
$$dP/dt = c_0P - d_0PR$$

$$dR/dt = c_1R + d_1PR$$

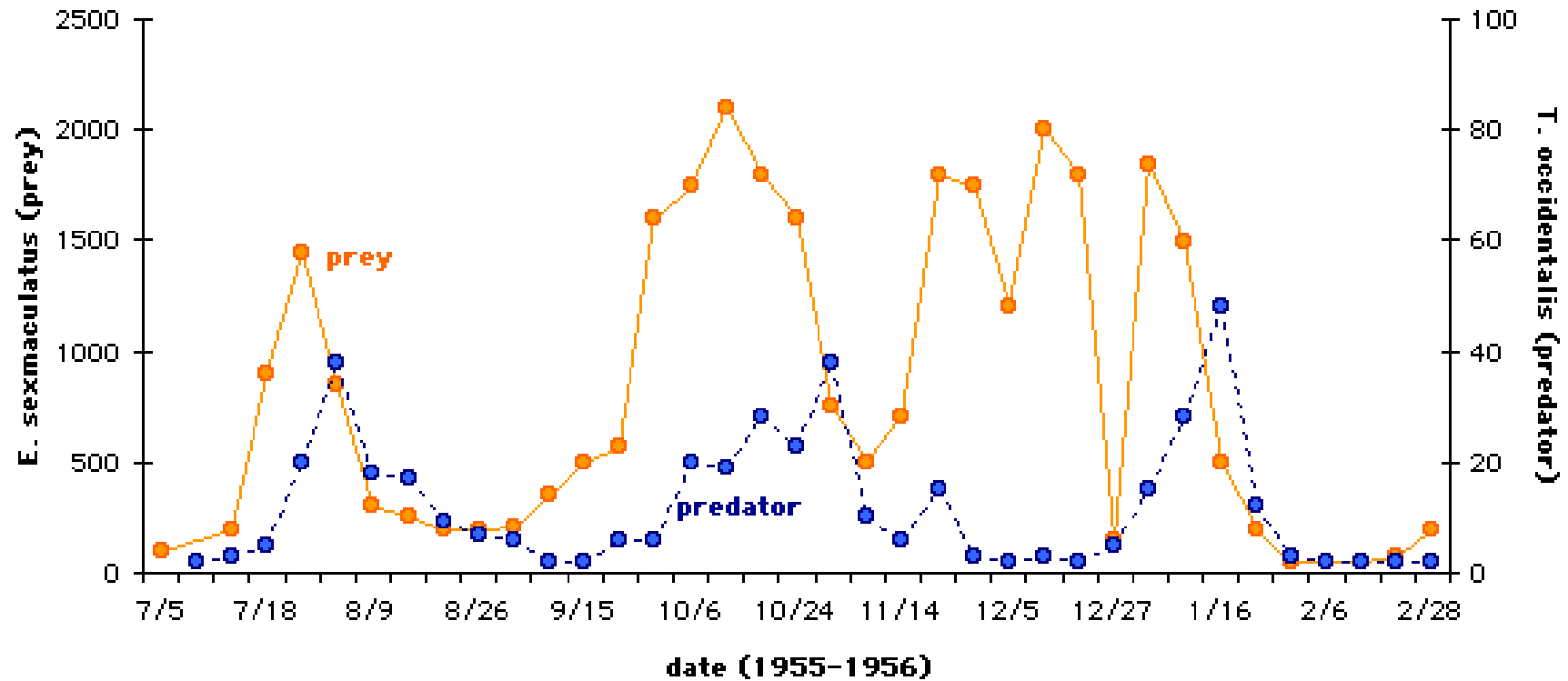


- Recibió una carta de Lotka sobre sus propias publicaciones previas.
- Se trata de modelos desarrollados independientemente.

Depredador-Presa – Ideal



Depredator-Presa – Real



Huffaker (1958)

LOS MUCHACHOS



- Alfred Lotka
(1880 – 1949)



- Vito Volterra
(1860 – 1940)

¿APLICACIONES EN A-LIFE?

- Son una herramienta para el estudio matemático de fenómenos emergentes a partir de componentes simples.
 - Se pueden definir “átomos” de un sistema a-life para analizar la emergencia de interacciones físicas
- Modelamiento de entidades biológicas, químicas.
- Farmacología
 - Morfogénesis química multicelular

UNA ENFERMEDAD VISTA CON UN AC

4 estados: Sano (S), Infectado1 (I1), Infectado2 (I2), Muerte (M)

Regla 1 – Si una celda S tiene al menos un vecino I1, o si tiene al menos 2 vecinos I2, entonces se transforma en I1.

En caso contrario, permanece en estado S.

Regla 2 – Una celda I1 se transforma en I2 después de 4 pasos (simulando semanas).

Regla 3 – Una celda I2 se transforma en una celda M.

Rule 4 – Una celda M se transforma en S, con probabilidad P1
se transforma en I1, con probabilidad P2

En caso contrario, permanece en estado M

AUTÓMATA CELULAR: VIDA CON REGLAS SIMPLES (UN CLÁSICO)

Tiburones y peces:

Relación depredador/presa

EJEMPLO

- Modelo de una relación depredador/presa
- Población inicial con distribución random de peces, tiburones y celdas vacías en una matriz de 1000 x 2000 celdas (2 millones de celdas)
- Inicialmente,
 - 50% de las celdas ocupadas por peces
 - 25% ocupadas por tiburones
 - 25% vacías

REGLAS

Alrededor de 12 reglas describen la vida en cada celda:

- nacimiento, longevidad y muerte de un pez o tiburón.
 - reproducción de peces y tiburones.
 - sobre – y baja – población.
 - interacción pez/tiburón.
-
- Importante: lo que ocurre en cada celda está determinado solamente por las reglas que se aplican localmente.

¡GRAN CANTIDAD DE COMPUTACIÓN!

- Aplica una docena de reglas a cada celda.
- ¡¡Para 2 millones de celdas!!
- Durante unas 20.000 generaciones.
- ¿Cuán rápido se puede hacer?

¡GRAN CANTIDAD DE COMPUTACIÓN!

- Usando un cluster de 20 CPU
- 15 veces más rápido que un solo PC
- Toma alrededor de 45 minutos
- ¡Es necesario introducir paralelismo!

REGLAS EN DETALLE: CONDICIONES INICIALES

Inicialmente las celdas contienen peces, tiburones o están vacías.

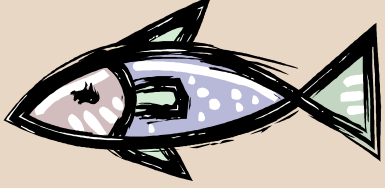
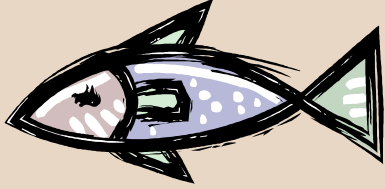
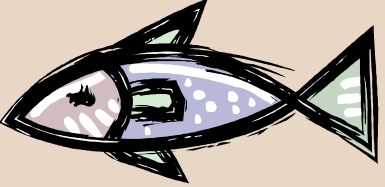
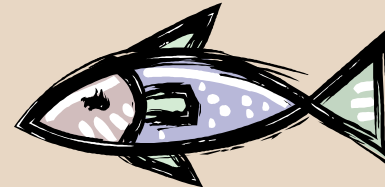
- Celdas vacías = 0 (pixel negro)
- Peces = 1 (pixel rojo)
- Tiburones = -1 (pixel amarillo)

REGLAS EN DETALLE: REPRODUCCIÓN

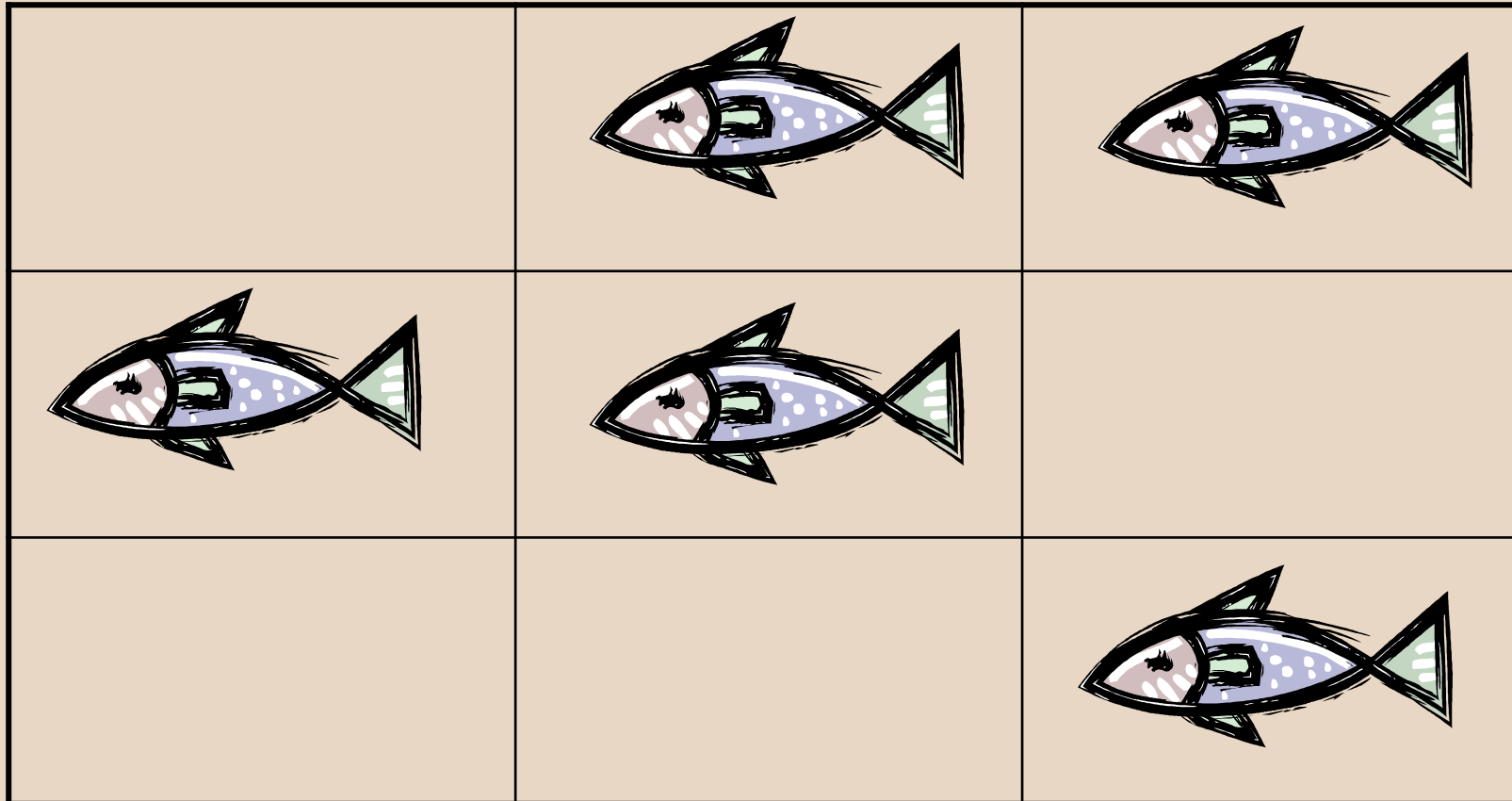
Regla de reproducción: si celda actual vacía

- Si hay ≥ 4 vecinos de una especie, y ≥ 3 de ellos están en edad de reproducción,
 - Edad reproducción pez ≥ 2 ,
 - Edad reproducción tiburón ≥ 3 ,
- Y hay < 4 de la otra especie:
- entonces crea una especie de ese tipo
- $+1$ = pez bebé (edad = 1 al nacer)
 - -1 = tiburón bebé (edad = $|-1|$ al nacer)

REGLA DE REPRODUCCIÓN: ANTES

		
	VACÍA	
		

REGLA DE REPRODUCCIÓN: DESPUÉS



REGLAS EN DETALLE: REGLAS PECES

Si la celda actual contiene un pez:

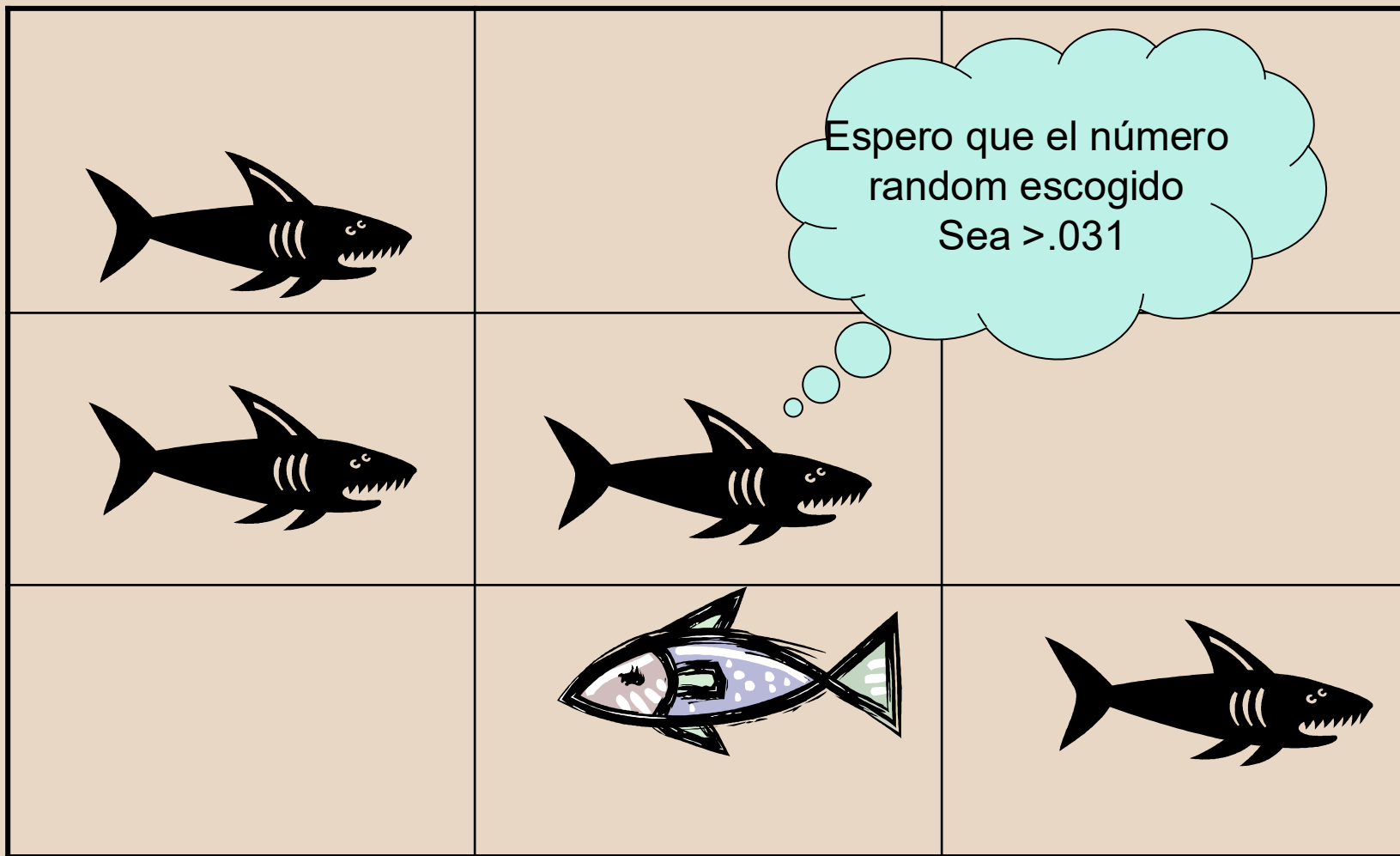
- Pez vive 10 generaciones.
- Si ≥ 5 vecinos son tiburones, el pez muere (comida del tiburón).
- Si los 8 vecinos son peces, el pez muere (sobrepoblación).
- Si el pez no muere, incrementar su edad.

REGLAS EN DETALLE: REGLAS TIBURONES

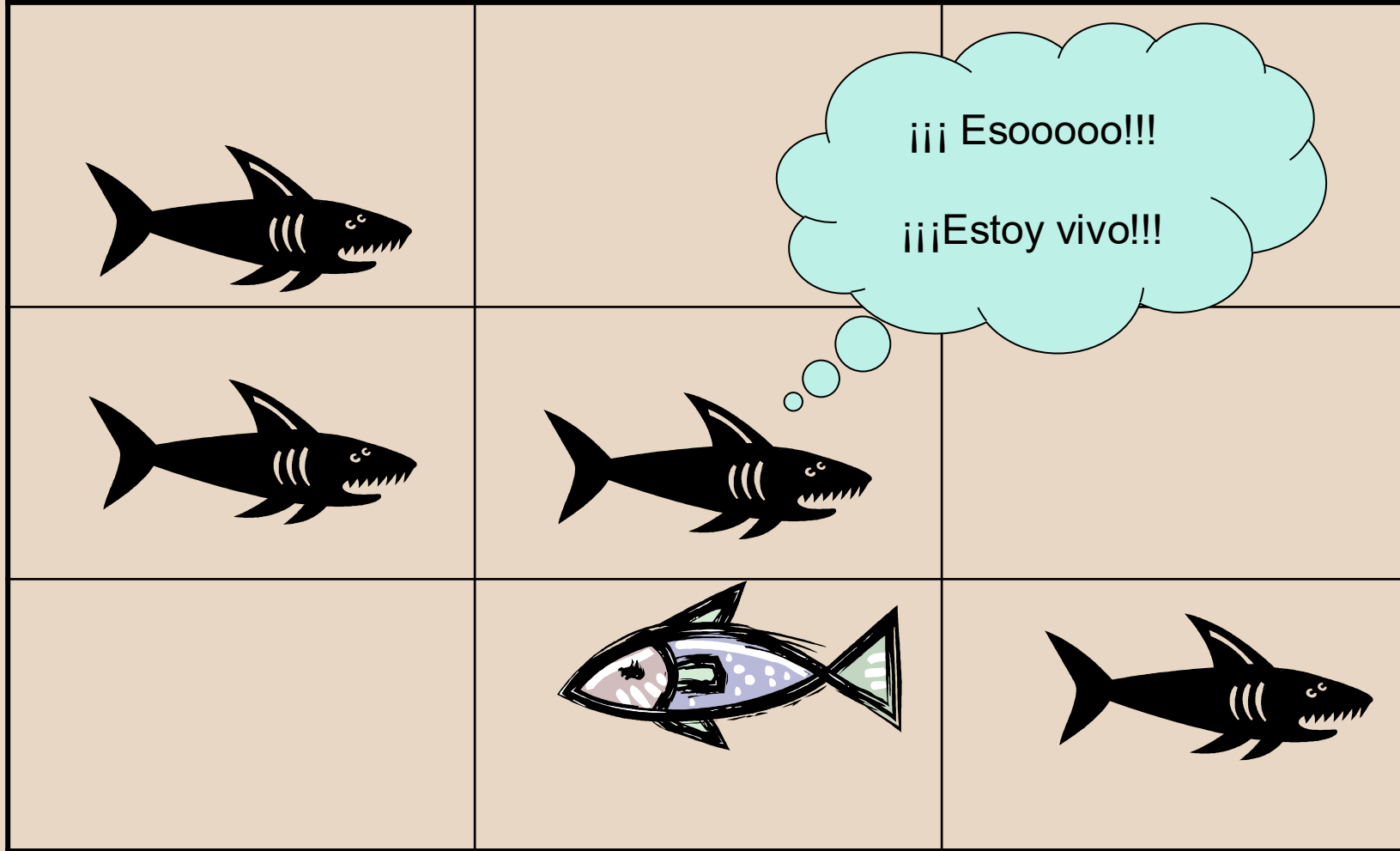
Si celda actual contiene un tiburón:

- Tiburón vive 20 generaciones
- Si ≥ 6 vecinos son tiburones y no hay peces vecinos, el tiburón muere (de hambre)
- Un tiburón tiene una posibilidad de $1/32$ (.031) de morir por causas diversas
- Si el tiburón no muere, incrementar su edad

MUERTE RANDÓMICA DEL TIBURÓN: ANTES



MUERTE RANDÓMICA DEL TIBURÓN: DESPUÉS



LÓGICA DE LA PROGRAMACIÓN

- Usar arreglos 2-dimensionales para representar la matriz
- En una posición (x, y) cualquiera, su valor:
 - Entero positivo (hay pez)
 - Entero negativo (hay tiburón)
 - Cero (celda vacía)
 - El valor absoluto de la celda es la edad

EJEMPLO DE CÓDIGO (C++): REPRODUCCIÓN

```
// If the current cell is empty, check to
// see if a shark or a fish is born
if (*current == 0)
{
    if ((fish >= 4) && (fish > sharks) && (fishbreed > 2))
        (*uu)(i,j) = 1;
    else
    {
        if ((sharks >= 4) && (sharks > fish) && (sharkbreed > 2))
            (*uu)(i,j) = -1;
        else
            (*uu)(i,j) = (*current);
    }
    continue;
}
```

PARALELISMO

- Una sola CPU hace todo esto:
 - Aplica reglas a la primera celda del arreglo.
 - Repite reglas para cada celda sucesivamente.
 - Después de procesar 2 millones de celdas, el arreglo se actualiza.
 - Ha pasado una generación.
 - Repite este proceso para muchas generaciones.
 - Cada 100 generaciones o algo así, convertir el arreglo a pixeles. rojos, amarillos y negros y enviar los resultados a pantalla

PARALELISMO

- Cómo distribuir el trabajo en 20 CPUs:
 - 1 CPU actúa como **master** (tiene copia del arreglo completo)
 - 18 CPUs actúan como esclavas (manejan partes del arreglo)
 - 1 CPU se encarga de actualizar la pantalla
- Problema: aspectos de comunicación entre CPUs esclavas relacionados con las celdas en los bordes del arreglo.

RESULTADOS

- Las próximas pantallas muestran el comportamiento sobre más de 10.000 generaciones (unos 25 minutos de procesamiento en el cluster)

TENDENCIAS DE LARGO PLAZO

- Las fronteras tienden a consolidarse en líneas horizontales, verticales y diagonales
- Se forman fronteras de celdas vacías entre las especies
- Grupos de peces tienden a unirse y generar formas convexas de 'comunidades'

VARIACIÓN DE CONDICIONES INICIALES

- (Todavía) usando distribución random:
 - Población media. Peces/Tiburones ocupan:
1/16 del total de la matriz
Peces: 62.703; Tiburones: 31.301
 - Poblaciones muy pequeñas. Peces/Tiburones ocupan:
1/800 del total de la matriz
Población inicial:
Peces: 1.298; Tiburones: 609

POBLACIONES MUY PEQUEÑAS

- La ubicación random de poblaciones muy pequeñas puede favorecer a una especie sobre la otra
- Peces favorecidos: tiburones mueren
- Tiburones favorecidos: predominan tiburones, pero los peces sobreviven en cantidades reducidas pero estables

POBLACIONES MUY PEQUEÑAS

- Los peces pueden vivir en comunidades estables aisladas de unos 20-30 individuos.
- Una comunidad con menos de 200 tiburones tiende a no ser viable.

COMPORTAMIENTO EN PARALELO

- Es posible una aceleración de hasta unas 18 veces.
- Normalmente: 15 veces más rápido.
- Razón: *overhead* de comunicación entre esclavos y master.
- El *overhead* de comunicación empeora si el intercambio de datos es frecuente.

THOMAS SCHELLING (1921 – 2016)

- Premio Nobel en Economía, año 2005
- Junto con Robert Aumann por "haber mejorado nuestra comprensión del conflicto y la cooperación a través del análisis de la teoría de juegos"

¿NOS INTERESA?

- Propuso un modelo para estudiar la segregación de grupos, a través las decisiones individuales, fomentadas por la preferencia de vivir con personas del mismo grupo.

¿NOS INTERESA?

- Mostró que dos grupos de personas pueden cambiar el entorno a un patrón altamente segregado sin una organización preestablecida.
- Su trabajo es relevante en el contexto de la modelación basada en agentes ya que las personas ejecutan reglas locales simples de interacción, tratan de satisfacer las restricciones que imponen en el entorno, y de manera autoorganizada surgen patrones globales de segregación.

HECHOS

- La evidencia empírica muestra que las personas intercambian información, conocimiento, bienes... **localmente.**
- Los AC permiten una representación de estructuras que interactúan de manera social o espacial.

DESCRIPCIÓN

- El modelo de segregación de Schelling puede estudiarse a través de la modelación basada en agentes.
- El modelo tiene dos posibles tipos de agentes, diferenciados por el color rojo y verde, posicionados en un entorno, los cuales actúan bajo información local para cumplir con sus requerimientos: estar con los de su mismo grupo (color).

DESCRIPCIÓN

- Si estas restricciones o requerimientos no pueden cumplirse entonces pueden moverse de un lugar a otro, hasta encontrar una localidad donde estén *cómodos* en su entorno.
- Es un sistema descentralizado, ya que no hay un agente líder que imponga ciertas conductas o que guíe el proceso (auto-organización).
- Las decisiones colectivas generan nuevas propiedades en el sistema en una escala global, llevando al sistema a exhibir patrones de segregación.

ELEMENTOS DEL MODELO

Agentes y entorno

1. Hay dos tipos de agentes en el entorno que toman decisiones con información local (rojo y verdes).
2. Los agentes constan de un modelo interno que busca satisfacer la necesidad de estar con los de su mismo tipo.
3. El entorno es homogéneo con un cierto porcentaje de celdas vacías.

ELEMENTOS DEL MODELO

Reglas

Los agentes consideran una vecindad de Moore (8-vecinos) para tomar una decisión en función del parámetro *porcentaje-de-similitud-requerida*.

1. Si el porcentaje de agentes de su mismo grupo cumple con el valor de similitud-requerida, entonces el agente permanece en su posición.
2. Si el porcentaje es menor, entonces el agente se cambia a otro lugar (celda vacía).

¿PODEMOS MODELAR LA PROPAGACIÓN DE UN CHISME?

- ¿Cómo lo haríamos?
- Se puede modelar hasta el impacto de ciertas modas...

SE HAN USADO AC PARA MODELAR FENÓMENOS MIGRATORIOS

- Reglas
 - Vecindad de Moore
 - 3 o más vecinos similares: permanece
 - Menos de 3 vecinos similares, se desplaza a una celda vacía

- Nos vemos el próximo lunes 😊
- Hablaremos sobre bacterias.
- Y en una de esas, si nos queda tiempo, de Teoría del Caos.